

MANCHESTER METROPOLITAN UNIVERSITY



FINAL YEAR DISSERTATION

A popup chatbot tutor to scaffold learning from video lectures and assess effectiveness

Author:
Hamzah Patel
22468473

Supervisor:
Keeley Crockett

Degree:
BSc (Hons) Computer Science

Department:
Computing and Mathematics

Faculty:
Science and Engineering

May 2026

Declaration

No part of this project has been submitted in support of an application for any other degree or qualification at this or any other institute of learning. Apart from those parts of the project containing citations to the work of others, this project is my own unaided work. This work has been carried out in accordance with the Manchester Metropolitan University research ethics procedures and has received ethical approval number **87671**.

Signed

Hamzah Patel

Acknowledgements

I am deeply thankful to my teachers for their expertise and to the participants for their essential contributions to this project. My heartfelt thanks also go to my friends and family for their constant encouragement during this milestone.

Abstract

Video-based learning is now central to higher education, but it can leave learners passive when lectures are watched without active retrieval, explanation, or feedback. Large language models create a related problem in the opposite direction: when used without educational guardrails, they can give direct answers that reduce the cognitive effort learning depends on. Existing AI-assisted video-learning systems address parts of this problem, but the reviewed literature did not identify a system that combines video-grounded retrieval, adaptive scaffolding, local deployment on consumer hardware, and human evaluation in one integrated tool.

This dissertation presents a locally deployed AI tutor designed to address that gap. The system uses Llama 3.1 8B with retrieval-augmented generation over video transcripts and supplementary readings, a five-level scaffolding engine that represents ZPD-inspired support as a per-session scalar score, and prompt-level guardrails that reduce direct answer extraction. The aim is to make video learning more active without allowing the tutor to do the learner's work.

Evaluation was conducted across three strands. In a partially counterbalanced within-subjects retention study with six participants, every participant showed a larger immediate retention gain from the topic studied with the AI tutor than from the topic studied in the video-only condition. The mean within-participant gain difference was 26.33 percentage points, with a one-sided paired Wilcoxon result of $p = 0.016$. RAGAS faithfulness averaged 0.747 across three runs, missing the 0.8 threshold and exposing a tension between useful pedagogical paraphrasing and strict chunk-level attribution; the secondary correctness criterion passed at 95.7%. Guardrail testing across 54 held-out prompts from three published attack taxonomies achieved 92.6% deflection against a 90% threshold.

The project demonstrates that a locally runnable, RAG-grounded tutor can improve immediate retention over passive video viewing in a small-scale human study, while also identifying clear limits around grounding, answer protection, and generalisability.

Table of Contents

DECLARATION	I
ACKNOWLEDGEMENTS	II
ABSTRACT	III
LIST OF FIGURES.....	VII
LIST OF TABLES	VIII
ABBREVIATIONS	IX
1. INTRODUCTION	1
1.1. PROJECT BACKGROUND	1
1.2. AIM AND OBJECTIVES	2
1.2.1. <i>Aim</i>	2
1.2.2. <i>Objectives</i>	2
1.2.3. <i>Research Questions and Hypotheses</i>	3
1.3. ETHICAL CONSIDERATIONS.....	3
1.4. DISSERTATION STRUCTURE	4
2. LITERATURE REVIEW.....	6
2.1. THE CHALLENGE OF GENERATIVE AI IN EDUCATION	6
2.2. THEORETICAL FOUNDATIONS: ZPD & SCAFFOLDED LEARNING.....	6
2.3. EXISTING APPROACHES TO AI-ASSISTED LEARNING.....	8
2.4. LIMITATIONS OF CURRENT APPROACHES	9
2.5. PROPOSED APPROACH AND TECHNICAL JUSTIFICATION.....	10
2.6. CONCLUSION	11
3. DESIGN	13
3.1. REQUIREMENTS.....	13
3.1.1. <i>Functional requirements: learner-facing</i>	13
3.1.2. <i>Functional requirements: teacher-facing</i>	14
3.1.3. <i>Non-functional requirements</i>	14
3.2. DEVELOPMENT METHODOLOGY	15
3.2.1. <i>Methodology by component type</i>	15
3.2.2. <i>RAG retrieval iteration</i>	16
3.2.3. <i>Guardrail iteration and tiered evaluation</i>	16
3.3. SYSTEM ARCHITECTURE	17
3.4. OPERATIONALISING THE ZONE OF PROXIMAL DEVELOPMENT (ZPD)	20
3.4.1. <i>Why a scalar</i>	20
3.4.2. <i>Why exponential smoothing</i>	20
3.4.3. <i>Why event-driven, not self-report</i>	21
3.5. SCAFFOLDING ENGINE DESIGN	21
3.5.1. <i>Five-level support model</i>	21
3.5.2. <i>Score update rule and event weighting</i>	22
3.5.3. <i>Initial level from the pre-quiz</i>	24
3.5.4. <i>State management and limitation</i>	24
3.6. RETRIEVAL-AUGMENTED GENERATION PIPELINE	25
3.6.1. <i>Corpus and ingestion</i>	25
3.6.2. <i>Embedding and storage</i>	26
3.6.3. <i>Retrieval and prompt assembly</i>	26

3.7.	GUARDRAIL DESIGN: CHAT CHANNEL.....	27
3.8.	TEACHER-FACING DESIGN: REDUCING FRICTION FOR ADOPTION	28
3.8.1.	<i>Motivation and scope</i>	28
3.8.2.	<i>The /admin interface: Questions, Participants, RAG debug</i>	28
3.8.3.	<i>Assistive question generation: layered grounding controls</i>	29
3.8.4.	<i>Observed failure modes and the case for human review</i>	29
3.8.5.	<i>Authorisation: a deferred concern</i>	30
3.9.	QUESTION STORE AND REUSE STRATEGY	30
3.10.	LEARNER-FACING USER INTERFACE	31
3.10.1.	<i>Layout and the interpolated question overlay</i>	31
3.10.2.	<i>Accessibility (with known limitations)</i>	31
3.11.	DEVELOPMENT TOOLS	32
3.12.	SYSTEM LIMITATIONS.....	32
4.	IMPLEMENTATION.....	34
4.1.	PROJECT SETUP AND BUILD	34
4.2.	STREAMING CHAT PATH	36
4.3.	QUESTION GENERATION.....	37
4.3.1.	<i>Pre-authored generation</i>	37
4.3.2.	<i>In-chat MCQ generation</i>	38
4.4.	TESTING.....	39
5.	EVALUATION	41
5.1.	EVALUATION CRITERIA	41
5.2.	AIMS AND OBJECTIVES REVIEW	42
5.2.1.	<i>Objective 1: Literature review</i>	42
5.2.2.	<i>Objective 2: System design</i>	42
5.2.3.	<i>Objective 3: System Implementation</i>	42
5.2.4.	<i>Objective 4: Evaluating Grounding</i>	43
5.2.5.	<i>Objective 5: Adversarial guardrail testing</i>	43
5.2.6.	<i>Objective 6: Within-subjects retention experiment</i>	43
5.2.7.	<i>Objective 7: Combined analysis of experiment and interaction-log data</i>	43
5.3.	RQ1 RETENTION	44
5.3.1.	<i>Experimental design</i>	44
5.3.2.	<i>Results</i>	44
5.3.3.	<i>Interpretation</i>	45
5.3.4.	<i>Limitations specific to RQ1</i>	45
5.4.	RQ2: FAITHFULNESS AND CORRECTNESS	45
5.4.1.	<i>Evaluation method</i>	45
5.4.2.	<i>Results</i>	46
5.4.3.	<i>Retrieval source composition</i>	46
5.4.4.	<i>Failure pattern and interpretation</i>	47
5.5.	RQ3: GUARDRAILS	47
5.5.1.	<i>Evaluation method</i>	47
5.5.2.	<i>Results</i>	48
5.5.3.	<i>Failure mode</i>	49
5.5.4.	<i>Interpretation</i>	49
5.6.	PARTICIPANT FEEDBACK	50
5.6.1.	<i>Participants</i>	50
5.6.2.	<i>Likert responses</i>	50
5.6.3.	<i>Free-text findings</i>	50
5.7.	LIMITATIONS OF THE EVALUATION	51

6.	CONCLUSION	53
6.1.	COMPLETED WORK	53
6.2.	SUMMARY OF FINDINGS AND LIMITATIONS.....	53
6.3.	FUTURE WORK	54
6.4.	REFLECTIONS	55
6.4.1.	<i>What worked and what did not</i>	55
6.4.2.	<i>Contributions from prior modules</i>	55
6.5.	CLOSING	56
7.	REFERENCES	57
8.	APPENDICES	62
8.1.	FEASIBILITY STUDY	62
8.1.1.	<i>Slide 1: Title</i>	62
8.1.2.	<i>Slide 2: Project Outline</i>	62
8.1.3.	<i>Slide 3: Project Fit</i>	62
8.1.4.	<i>Slide 4: Aims and Objectives</i>	63
8.1.5.	<i>Slide 5: Research Context 1</i>	63
8.1.6.	<i>Slide 6: Research Context 2</i>	64
8.1.7.	<i>Slide 7: Research Methodology</i>	64
8.1.8.	<i>Slide 8: Implementation Methodology</i>	65
8.1.9.	<i>Slide 9: Resources</i>	65
8.1.10.	<i>Slide 10: Project Plan</i>	66
8.1.11.	<i>Slide 11: Potential Risks and Solutions</i>	66
8.1.12.	<i>Slide 12: Project Impact considerations</i>	67
8.1.13.	<i>Slide 13: Project Evaluation Plan</i>	67
8.1.14.	<i>Slide 14: In Conclusion</i>	68
8.1.15.	<i>Slide 15: References</i>	68
8.2.	TUTOR FLOW DIAGRAM.....	70
8.2.1.	<i>Setup, pre quiz, video loop and post-quiz path</i>	70
8.2.2.	<i>Interpolated MCQ handling, RAG retrieval, scaffold adjustment and answer handling.</i>	71
8.2.3.	<i>Response evaluation branch and closing MCQ entry.</i>	72
8.2.4.	<i>Question classification, support strategy and tutor response branch</i>	73
8.2.5.	<i>Hint-level selection and support strategy branch</i>	74
8.2.6.	<i>Closing MCQ, student answer and discussion decision</i>	75
8.2.7.	<i>Session closure and loop back to video watching</i>	76
8.3.	ETHICS APPROVAL: TITLE, NUMBER AND DATE	77
8.4.	README PROJECT SETUP	77
8.5.	POST SESSION QUESTIONNAIRE (MICROSOFT FORMS)	78
8.6.	QUESTION GENERATION USER PROMPT	80
8.7.	GROUNDING REVIEWER PROMPT	81
8.8.	PER-EXCERPT GENERATION LOOP	82
8.9.	STREAMING CHAT HANDLER AND CITATION REPAIR	85
8.10.	BUFFERED CHAT RETRY PATH.....	90
8.11.	SEP SOURCES JSON	91
8.12.	CONSENT FORM	92
8.13.	PARTICIPANT INFORMATION SHEET	94
8.14.	ANONYMISED LIKERT QUESTIONNAIRE RESPONSES.....	96
8.15.	ANONYMISED FREE-TEXT QUESTIONNAIRE RESPONSES.....	96

List of Figures

Figure 1: System Architecture	18
Figure 2: Initial wireframe	31
Figure 3: Welcome surface at session start, showing participant ID, group setup selection, model selection and consent checkboxes.....	35
Figure 4: Streamed tutor reply with the inline citation marker [1] resolving to a transcript chunk shown in the citation block beneath	37
Figure 5: Remediation MCQ card rendered in the chat panel.	38

List of Tables

Table 1: Research questions and hypotheses.	3
Table 2: Three-tier struggle hierarchy for assisted learning	8
Table 3: Comparison of related works against identified research gap	9
Table 4: Scaffold score level thresholds	22
Table 5: Score update rule after correct interpolated MCQ.....	23
Table 6: Different MCQ events with update rules	23
Table 7: Counterbalancing groups for the within-subjects retention experiment.	44
Table 8: Per-participant retention results across AI-tutor and video-only conditions.	45
Table 9: RAGAS faithfulness and correctness results across three runs	46

Abbreviations

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
ARIA	Accessible Rich Internet Applications
BY-NC-SA	Attribution-NonCommercial-ShareAlike
COVID-19	Coronavirus Disease of 2019
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DRF	Django REST Framework
FR-L	Functional Requirement for Learners
FR-T	Functional Requirement for Teachers
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
ID	Identifier
ITS	Intelligent Tutoring System
JSON	JavaScript Object Notation
LLM	Large Language Model
MCQ	Multiple Choice Question
NDJSON	Newline Delimited JSON
NFR	Non-functional Requirement
OCR	Optical Character Recognition
RAG	Retrieval-Augmented Generation
RAGAS	Retrieval-Augmented Generation Assessment
REST	Representational State Transfer
RQ	Research Question
SEP	Stanford Encyclopedia of Philosophy
STEM	Science, Technology, Engineering and Mathematics
UI	User Interface
URL	Uniform Resource Locator
VRAM	Video Random Access Memory
VTT	Web Video Text Tracks
ZPD	Zone of Proximal Development

1. Introduction

1.1. Project Background

Video-based learning, broadly defined as the use of recorded or streamed video as a primary instructional medium, has long been part of higher education, but its adoption accelerated during the COVID-19 pandemic as institutions shifted to remote delivery through platforms such as Zoom and Microsoft Teams (Lockee, 2021). While video is not inherently detrimental to learning, it can encourage passive consumption when learners watch without retrieval, questioning, or explanation. Learners may complete a lecture believing they have understood material they have only observed, making active engagement central to the educational value of the medium (Deslauriers et al., 2019).

The same problem is intensified by the way students now use generative AI for academic support. Since the release of ChatGPT in late 2022, students have increasingly used these tools for instant, personalised academic support (Singer-Freeman et al., 2025). However, large language models are often optimised to produce helpful and satisfying responses, which can mean giving direct explanations rather than requiring the learner to reason through the problem (Cohn et al., 2026). Used alongside passive video learning, this risks replacing productive cognitive effort with an illusion of competence: the learner receives an answer, but may not do the work needed to retain or apply the concept.

This problem was not only theoretical for the present project. As a computing student, much personal study, including revision, relied on video lectures and recorded explanations. These resources were valuable, but they exposed a recurring problem: it is easy to feel that learning is taking place simply because material is being watched, even when little active recall, application, or explanation is occurring. This was sharpened by a wider digital environment in which short-form content competes for attention and sustained focus feels harder to maintain. The project therefore became personally meaningful: it investigates whether AI support can keep learners active rather than letting them mistake exposure for understanding.

Educational theory frames this as a scaffolding problem. Vygotsky's (1978) Zone of Proximal Development describes the gap between what a learner can do alone and what they can do with support, while Wood, Bruner and Ross (1976) describe scaffolding as temporary assistance adjusted to learner performance. A standard large language model has no built-in mechanism for making this adjustment, so this project develops a locally deployed tutoring system that grounds responses in the material being studied and uses a lightweight rule-based learner model to adjust support during the session. Its effectiveness is evaluated through a within-subjects experiment comparing immediate retention between AI-assisted and video-only learning.

1.2. Aim and Objectives

1.2.1. Aim

This project aims to develop and evaluate a locally deployable, RAG-grounded LLM tutoring system that supports video-based learning through ZPD-inspired adaptive scaffolding, measuring its impact on immediate retention compared with video-only learning.

1.2.2. Objectives

The objectives of this project are as follows:

- Review literature on ZPD-based scaffolding, the challenges of generative AI in educational contexts, existing approaches to AI-assisted video-based learning, and the technical foundations underpinning local LLM deployment and retrieval-augmented generation
- Design a system architecture integrating a local LLM with RAG over video transcripts, interpolated knowledge checks that prompt learner engagement, and pedagogical guardrails to prevent direct answer extraction
- Implement the system using Ollama, with a pre-quantised local LLM suitable for consumer hardware constraints
- Evaluate response faithfulness and correctness using the RAGAS framework

- Conduct adversarial testing to evaluate guardrail robustness against direct answer extraction
- Design and conduct a within-subjects experiment comparing immediate retention scores between the AI-assisted and video-only conditions
- Analyse experimental results alongside interaction log data to evaluate system effectiveness

1.2.3. Research Questions and Hypotheses

This project investigates the following research questions:

	Research Question	Hypothesis
1	Does the RAG-grounded adaptive tutor improve immediate retention compared with video-only learning?	Post-quiz improvement will be greater in the AI-tutor condition than in the video-only condition.
2	Does the tutor maintain faithful and correct responses using retrieved content?	Mean RAGAS faithfulness will exceed 0.8, with correctness reported against a secondary target of 0.9.
3	How effective are the system's guardrails against direct answer extraction?	Guardrail deflection rate will exceed 90% against a predefined adversarial prompt set.

Table 1: Research questions and hypotheses.

1.3. Ethical Considerations

This project received ethical approval from Manchester Metropolitan University (EthOS reference: 87671). All participants were provided with a participant information sheet and consent form before taking part; both documents are included in Appendix 8.12 and Appendix 8.13. Consent and eligibility were also enforced in the prototype: participants could not start the session until they had confirmed that they had read the information sheet, were over 18, were a university STEM student, had read and accepted the consent form, and understood that the post-session questionnaire was optional. Participation was voluntary, and participants were informed of their right to withdraw without giving a reason until 28 March 2026. After that date, withdrawal was no longer possible because the research data was stored without direct identifiers and no formal linking record was retained.

The study collected task-performance and interaction data, including quiz scores, response correctness, hint usage, chat logs, and interaction timing data. Personal details were collected only for consent purposes and stored separately from the research dataset, which contained no names or direct identifiers. Data were stored on password-protected systems accessible only to the research team, with a secure backup on university OneDrive. The consent form also gave permission for questionnaire responses and AI-interaction data to be deposited in OneDrive for future research and learning.

The project used third-party learning materials, including Wireless Philosophy videos and Stanford Encyclopedia of Philosophy entries, as source material for retrieval and evaluation. The two Wireless Philosophy videos used in the study, Liberalism and Representative Democracy, are also available as Khan Academy partner content. Khan Academy’s usage guidance states that its website materials, including videos, may be used under Creative Commons BY-NC-SA terms, subject to attribution and non-commercial use (Khan Academy, 2021). The Stanford Encyclopedia of Philosophy terms permit individual users to read, download, make copies, print, search and link to entries, subject to non-commercial and limited-distribution conditions (Stanford Encyclopedia of Philosophy, no date). Source attribution was retained in the report and citation metadata was surfaced in the system where relevant. Only the material required for the study was ingested into the local corpus.

The system included light, dark, high-contrast, and colourblind display modes as initial accessibility provisions. However, the learner-facing colourblind mode contained a known contrast defect, discussed in Chapter 3. A comprehensive accessibility audit, including screen-reader compatibility and full keyboard navigation, was beyond the scope of this implementation and is identified as future work.

As discussed in Section 2.2, the tutoring system incorporates pedagogical guardrails intended to reduce direct answer extraction, supporting learning through guided interaction rather than solution delivery. This is relevant to academic integrity because the tool was intended to support understanding rather than enable the circumvention of independent learning.

1.4. Dissertation Structure

The remainder of this dissertation is structured as follows. Chapter 2 reviews the literature on scaffolding, generative AI in education, video-based learning, and retrieval-augmented generation, identifying the research gap addressed by the project. Chapter 3 presents the system design, including the architecture, scaffolding engine, RAG pipeline, guardrails, and user interfaces. Chapter 4 describes the implementation and testing approach. Chapter 5 evaluates the system against the research questions through the retention study, RAGAS evaluation, guardrail testing, and participant feedback. Chapter 6 summarises the findings, limitations, future work, and reflections.

2. Literature Review

2.1. The Challenge of Generative AI in Education

Since the release of ChatGPT in 2022, learners have increasingly used generative AI for academic support (Singer-Freeman et al., 2025; Adiguzel et al., 2023; Kasneci et al., 2023). These tools offer instant, personalised explanations, but unrestricted use risks removing the productive struggle that learning requires (Barcaui, 2025). This risk reflects a wider pattern of cognitive offloading, where people rely on external aids rather than internal recall or problem solving. The “Google effect”, for example, describes people remembering where to find information rather than the information itself (Sparrow, Liu and Wegner, 2011). Risko and Gilbert’s (2016) metacognitive model similarly explains offloading as a response to perceived internal effort. In generative AI use, this can shift effort away from problem solving and critical evaluation towards output verification and integration (Lee et al., 2025). This matters because the generative processes being bypassed are often the “desirable difficulties” that support long-term retention and transfer (Bjork and Bjork, 2011).

Video-based learning intensifies this problem. Its use in education expanded during the COVID-19 pandemic (Lockee, 2021; Mujallid, 2024), but video can promote passive engagement when learners watch without active retrieval or explanation. Was, Hollis and Dunlosky (2019) found that students reported task-unrelated thoughts more than half the time when prompted during online video lectures. This passivity can create an illusion of understanding, where learners believe they have understood material they have only observed (Deslauriers et al., 2019; Hörnlein and Kulgemeyer, 2025). When a video then prompts a question that requires genuine engagement, generative AI may become the path of least resistance, reducing effort while weakening depth of learning (Stadler, Bannert and Sailer, 2024). The central question is therefore how AI can be integrated into video-based learning so that it scaffolds understanding without bypassing the productive struggle that makes learning durable.

2.2. Theoretical Foundations: ZPD & Scaffolded Learning

The pedagogical concept underpinning this project is Vygotsky’s (1978) Zone of Proximal Development (ZPD), defined as the gap between what a learner can achieve independently and what they can achieve with support from a more knowledgeable other.

Instruction is most effective within this zone because it targets developing capabilities: assistance beyond the learner’s reach is ineffective, while assistance for what they can already do independently offers little cognitive growth. Wood, Bruner and Ross (1976) describe this temporary, structured support as scaffolding, where task demands are reduced to a manageable level and support is gradually withdrawn as competence develops. Bjork and Bjork (2020) echo this calibration principle, arguing that difficulties only become “desirable” when matched to prior knowledge. A standard large language model, however, has no inherent mechanism for assessing learner competence or withdrawing support; without deliberate design, it tends to provide maximum assistance regardless of need.

Miscalibrated scaffolding fails in two directions. Over-scaffolding removes the cognitive effort that learning requires. Large language models are prone to this because their training often rewards helpful, complete responses rather than productive restraint. OpenAI (2025) acknowledged a related issue in models that prioritised pleasing users over genuinely helping them, while Cohn et al. (2026) argue that satisfaction-oriented LLM training can limit opportunities for critical thinking and enable “prompt hacking”, where students learn to extract answers rather than engage with material. However, simply restricting AI output creates the opposite risk. Under-scaffolding leaves learners without enough support, turning desirable difficulty into frustration (Bjork and Bjork, 2020). In Vygotskian terms, this pushes the learner beyond the ZPD. The challenge is therefore not to eliminate AI assistance, but to calibrate it: providing enough support to sustain engagement while preserving the effort learning requires.

This produces a three-tier view of assisted learning. When task demands fall within the learner’s ZPD, struggle is productive. When demands exceed it, difficulty becomes destructive. When support removes difficulty entirely, the learner experiences false fluency: an illusion of competence sustained by cognitive offloading rather than understanding. This distinction provides the analytical basis for the project’s scaffolding design. Since an LLM cannot naturally judge where the learner sits within these tiers, the system requires a proxy for learner state, addressed in Chapter 3.

Tier	Mode	Relation to ZPD	Scaffolding failure	Learning outcome
1	Productive struggle	Within	Appropriate	Understanding

2	Destructive difficulty	Beyond	Insufficient	Frustration
3	False fluency	Below	Excessive	Illusion of competence

Table 2: Three-tier struggle hierarchy for assisted learning

2.3. Existing Approaches to AI-Assisted Learning

Efforts to use computers to replicate human tutoring gained traction in the 1980s, including Carnegie Mellon’s rule-based cognitive tutors for domains such as geometry and LISP (Anderson et al., 1995). These systems, now classed as Intelligent Tutoring Systems (ITS), provided immediate feedback but required extensive domain-specific engineering. This limitation has driven interest in general-purpose large language models for educational support.

Khan Academy’s Khanmigo uses an LLM to guide learners through Socratic questioning rather than simply providing direct answers (Khan Academy, 2023). A course-scale example is CS50.ai, where Liu et al. (2024) deployed the GPT-4-based “CS50 Duck” in Harvard’s CS50 with course-specific prompts, lecture-caption RAG, and pedagogical guardrails. These systems demonstrate the value of guarded LLM support in education, but they do not address the specific combination targeted here: video-based retention, local deployment, and explicit learner-state tracking for adaptive scaffolding.

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2021), addresses the grounding limitation by retrieving content from a specified knowledge base at inference time and incorporating it into the model’s response. In education, this allows a tutor to ground responses in the material the learner is actively studying, such as a video transcript or supplementary course readings, rather than relying on parametric knowledge alone.

Recent video-learning systems address parts of this problem. Rumble et al. (2026) present LEA, a multi-agent framework combining course-specific RAG, cognitive load estimation, ZPD inference, and motivation tracking, but evaluate it only with simulated learner agents. Abraar et al. (2026) embed a timestamp-aligned RAG conversational tutor alongside video lectures and report improved learning performance in a sequential within-subjects study ($N = 58$; $d = 1.505$), but the system is API-dependent, lacks an explicit learner model, and may be affected by fixed-order practice effects. Hobert (2025) constructs a RAG knowledge base from lecture recordings using OCR and speech-to-

text, but evaluates the prototype only from educator and designer perspectives. Islam et al. (2025) demonstrate local deployment with ALIVE, combining timestamp-aligned retrieval and a locally hosted language model for real-time lecture interaction, but do not operationalise ZPD-calibrated scaffolding or adaptive learner-state tracking.

This project therefore focuses on the intersection these works only partially address: video-grounded RAG, explicit ZPD-calibrated scaffolding, local deployment on consumer hardware, and human evaluation.

System	Video-grounded RAG	ZPD-calibrated scaffolding	Local deployment	Human Evaluation
CS50.ai (Liu et al., 2024)	Yes	No	No	Yes
LEA (Rumble et al., 2026)	Yes	Yes	Not specified	No (simulated)
Abraar et al. (2026)	Yes	No	No	Yes
Hobert (2025)	Yes	No	Not specified	Educators only
ALIVE (Islam et al., 2025)	Yes	No	Yes	No
This project	Yes	Yes	Yes	Yes

Table 3: Comparison of related works against identified research gap

Note. “Not specified” means the deployment model is not explicitly reported. “Simulated” means ZPD inference was implemented but validated with simulated learner agents rather than human participants. CS50.ai is included as a course-grounded LLM deployment rather than a video-learning system in the same sense as this project.

2.4. Limitations of Current Approaches

The comparison table shows that recent systems address important parts of the problem, but not the full combination required here. The reviewed work demonstrates video-grounded retrieval, conversational support, local deployment, or human evaluation, but none combines these with explicit scaffolding calibrated to learner performance in a locally deployable video-learning system.

Two gaps are most relevant. First, learner-state tracking is often unclear or under-specified. Abraar et al. (2026), for example, describe adaptive formative assessment but do not detail a persistent mechanism for tracking learner state across the session. Second,

some systems remain primarily learner-initiated. ALIVE (Islam et al., 2025) embeds support within lecture playback through timestamp-aligned retrieval, but may still struggle to address false fluency or passive disengagement when learners do not recognise their own uncertainty.

Deployment is also a constraint. Rumble et al. (2026) and Hobert (2025) do not clearly specify locally deployable implementations, while Abraar et al. (2026) use an API-based architecture. This limits suitability where privacy, cost, and connectivity are practical barriers. These limitations motivate the proposed approach in Section 2.5.

2.5. Proposed approach and technical justification

The limitations identified in the literature motivate a system that combines video-grounded retrieval, session-level scaffolding, local deployment, and small-scale human evaluation within a single workflow. The approach is not to replace video learning, but to make it more active: the learner still watches the lecture, but the system interrupts passivity through interpolated checks, provides tutor support through chat, and adjusts assistance using observable interaction events. Since interpolated questions create an obvious route for answer extraction, the approach also requires guardrails that support conceptual help without revealing MCQ answers.

The grounding mechanism is Retrieval-Augmented Generation. RAG augments generation by retrieving relevant external documents at inference time, reducing reliance on the model’s parametric knowledge alone (Lewis et al., 2021). In this project, the retrieved material consists of video transcript chunks and supporting philosophical sources. This addresses hallucination and relevance concerns because responses are shaped by the material selected for the course rather than by training data alone. RAG is also more appropriate than fine-tuning because the knowledge base changes with each video; fine-tuning would require model updates for each new item of content, while RAG allows new material to be ingested without retraining the model.

The scaffolding mechanism is intentionally lightweight. Instead of attempting a full cognitive model, the system maintains a session-scoped estimate of support need based on pre-quiz performance and observable interaction events during the video. This draws on the student-modelling role of intelligent tutoring systems, but simplifies it into a transparent rule-based proxy suitable for a single-session study (VanLehn, 2011).

Interpolated questions serve two roles. Pedagogically, they interrupt passive viewing; prior work shows that interspersed tests during video lectures can reduce mind wandering, improve retention, and improve calibration between predicted and actual performance (Szpunar, Khan and Schacter, 2013; Szpunar, Jing and Schacter, 2014). Architecturally, they provide the entry point into the tutor: a correct answer can trigger explanation and consolidation, while an incorrect answer can trigger remediation. The same mechanism therefore both prompts active engagement and supplies evidence for adapting support.

Local deployment is retained as a core technical constraint. It avoids per-query API costs, reduces dependence on external services during participant sessions, and keeps learner interaction data on the local machine. This requires an open-weight, instruction-tuned model that can run within consumer hardware limits after quantisation. Candidate models in the 7–9B parameter range were compared using Open LLM Leaderboard v2 scores, context-window size, Ollama availability, and expected fit within the 12GB VRAM constraint. The leaderboard was used as secondary evidence because the tutoring system required deployability and context capacity, not only high general benchmark scores (Hugging Face, no date). Other candidates, including Gemma 2 9B IT and Phi-4-mini-instruct, were considered, but Llama 3.1 8B Instruct offered the best fit for the project’s combination of Ollama support, local deployability, and context-window capacity.

The selected model was deployed through Ollama in a 4-bit quantised format. Quantisation reduces memory requirements, enabling local inference on consumer hardware; prior work has shown that carefully designed 4-bit quantisation can preserve much of a language model’s task performance while substantially reducing memory cost (Dettmers et al., 2023). This made the system feasible on the target laptop without relying on an external API.

2.6. Conclusion

This chapter situated the project within the challenge of integrating AI into video-based learning without undermining productive struggle. It argued that unrestricted generative AI can encourage cognitive offloading, that video-based learning can promote passive engagement, and that miscalibrated scaffolding can produce false fluency or destructive difficulty rather than understanding. Existing systems address parts of this problem, but

none reviewed here combines video-grounded retrieval, session-level scaffolding, local deployment on consumer hardware, and human evaluation within a single artefact.

The proposed system addresses this gap through a locally deployed RAG tutor with interpolated checks, adaptive scaffolding, and answer-extraction guardrails. Chapter 3 describes its design in detail.

3. Design

This chapter presents the design of the AI-tutoring system, covering its requirements, development methodology, architecture, subsystem designs, trade-offs, and limitations.

3.1. Requirements

Requirements were derived from Chapter 2's pedagogical theory, the Chapter 1 research questions, and the constraints of local deployment. Functional requirements are divided into learner-facing and teacher-facing groups; non-functional requirements apply system-wide.

3.1.1. Functional requirements: learner-facing

Learner-facing requirements cover the participant's tutored session, from pre-quiz through video interaction to post-quiz.

- **FR-L1, Video-grounded tutoring.** The system must let learners watch a video and ask questions about its content, returning answers grounded in the transcript and/or supporting reference material rather than the model's prior knowledge.
- **FR-L2, Adaptive scaffolding.** The system must dynamically adjust the support during a session, increasing autonomy as the learner demonstrates understanding and providing more direct support when they struggle. The adjustment must be driven by interaction events, not self-report.
- **FR-L3, Pre/post assessment on the same item set.** The system must administer a pre-quiz before tutoring and a post-quiz after, on both topics that the user watches videos on: Liberalism and Representative Democracy, drawn from the same question set, enabling immediate retention to be measured by within-subjects comparison (RQ1).
- **FR-L4, Interpolated comprehension checks.** The system must interrupt the video lecture with multiple-choice questions that test understanding rather than relying on learners to identify their own gaps.
- **FR-L5, Citation and traceability.** Tutor responses derived from source material must indicate which sources were used. Citations originating from a video chunk must allow the learner to seek the video to the relevant moment in a single action.

- **FR-L6, Session logging.** Consent records, pre/post quiz scores, chat interactions and interpolated MCQ outcomes must be stored for potential later analysis.
- **FR-L7, Adversarial robustness.** The chat channel must resist attempts to bypass the tutoring role and produce off-task content (RQ3).

3.1.2. Functional requirements: teacher-facing

Teacher-facing requirements reduce developer dependence by supporting content loading, question review, corpus inspection, and data export.

- **FR-T1, Content ingestion.** Teachers must be able to add written reference material (by URL) and video transcripts (as .vtt files) to the grounding corpus without writing code or editing files on disk.
- **FR-T2, Assistive question generation.** The system must generate candidate MCQs for pre-quiz, post-quiz, and interpolated-check use from ingested transcripts, with teacher editing before finalisation.
- **FR-T3, Teacher review and editing.** Teachers must be able to review, edit, and delete generated and stored questions through the admin interface. The design supports post-generation teacher correction, but does not enforce mandatory approval before learner exposure.
- **FR-T4, Participant data inspection.** Teachers must be able to view participant records and export them for analysis.
- **FR-T5, Corpus inspection.** Teachers must be able to inspect the grounding corpus to diagnose retrieval problems and verify that uploaded material is reaching the vector store.

3.1.3. Non-functional requirements

Non-functional requirements define system properties rather than user-facing features.

- **NFR-1, Local deployment.** The core tutoring system must run on a single machine without external model, retrieval or generation API calls during participant sessions.

- **NFR-2, Faithfulness and correctness.** Tutor responses should remain grounded in the retrieved corpus, targeting RAGAS faithfulness above 0.8 and correctness above 0.9 (RQ2).
- **NFR-3, Guardrail performance.** Adversarial deflection on the chat channel should reach at least 90% across a prompt set derived from established taxonomy (RQ3).
- **NFR-4, Accessibility provision.** The system should provide theme variants (light, dark, high-contrast) and colourblind-mode overrides, with the goal of making the interface usable by participants with common forms of colour-vision deficiency.
- **NFR-5, Comprehensibility for participants.** The interface must be intuitive enough to be learnable in the short on-boarding period available in a within-subjects study, without dedicated training.
- **NFR-6, Reproducibility.** Model versions, retrieval parameters, prompt templates and scaffold-event weights must be fixed and recorded, so that the evaluated system can be reconstructed.
- **NFR-7, Teacher review support.** Generated questions must be reviewable and editable by a teacher. The current implementation supports review and correction but does not enforce publication gating.

3.2. Development methodology

3.2.1. Methodology by component type

The system is built from two kinds of components: stable parts where the requirements are well understood up-front, and uncertain components whose behaviour had to be discovered through testing. The project therefore adopted a hybrid methodology: incremental Waterfall delivery for stable software components, and an iterative prototyping for uncertain LLM-dependent components. This approach was set out in the feasibility study (Appendix 8.1.8) and then applied during implementation, although the relative ordering of the two streams ran partly in parallel.

- **Iterative prototyping for uncertain parts.** This covered the RAG retrieval pipeline, the tutor prompt design, and guardrail addendum. The depth of iteration

differed between components. In practice this work was substantially more informative once the running system existed around it, because the model's behaviour under a given prompt depended on the surrounding context (chat history, retrieved chunks, scaffold level) in ways that isolated prompt-testing could not reproduce.

- **Incremental Waterfall delivery for stable parts.** This covered the React frontend skeleton, the Django REST endpoints, the SQLite schema, the video player, the chat panel, the quiz routing and the session-logging code. These requirements were sufficiently clear to support a sequenced build: interface skeleton, data flow, then logging.

The hybrid approach matched the contrast between stable software components and uncertain LLM behaviours. UI scaffolding is largely binary: a chat panel either functions correctly or it does not. LLM behaviour, however, must be tested empirically, because a system can execute successfully while still producing pedagogically inappropriate, ungrounded, or unsafe responses. A linear approach would therefore have been unsuitable for LLM-dependent components, while repeated UI iteration would have added little value.

3.2.2. RAG retrieval iteration

RAG parameters, including chunk size, overlap, and retrieval distance cutoff, were tuned informally using sample questions during development rather than against a fixed evaluation set. Overlap received the most iteration, with successive values tested by inspecting which chunks were retrieved for borderline queries. Final faithfulness and correctness scoring against the 47-question RAGAS evaluation set was conducted during evaluation rather than used as a design feedback signal.

3.2.3. Guardrail iteration and tiered evaluation

The guardrail loop used a deliberately small development set of five hand-written prompts, each instantiating a distinct prompt-hacking category: direct extraction, character role-play, assumed responsibility, sudo-mode escalation, and encoded-output requests. These categories were adapted from Liu et al.'s (2023) empirical taxonomy of jailbreak prompt patterns and informed by Wei et al.'s (2023) account of safety-training failure modes. The development set was intended to expose the

structural shape of each attack type without encouraging overfitting. Once the guardrail addendum deflected all five prompts, evaluation moved to held-out testing.

Two held-out test sets were used. The first contained 39 prompts across thirteen attack categories adapted from the same Liu- and Wei-derived prompt set. After strong performance on this set, a second held-out test was added to push the evaluation further: 15 prompts across five non-overlapping attack categories from Schulhoff et al. (2024). Together, the three sets formed a tiered evaluation: the development set drove design, the broad held-out set tested coverage, and the disjoint held-out set tested generalisation. This tested whether the guardrail handled unseen attack patterns, not only those it was designed against.

3.3. System architecture

The system is deployed as a single-machine application with five logical layers: web frontend, backend application, RAG orchestration, local model-serving, and persistent storage layer split between a vector store and a relational database. All components communicate over localhost, with no external API calls during participant sessions. This satisfies NFR-1 and supports NFR-6 by fixing the system to a local configuration. Figure 1 below shows the full architecture.

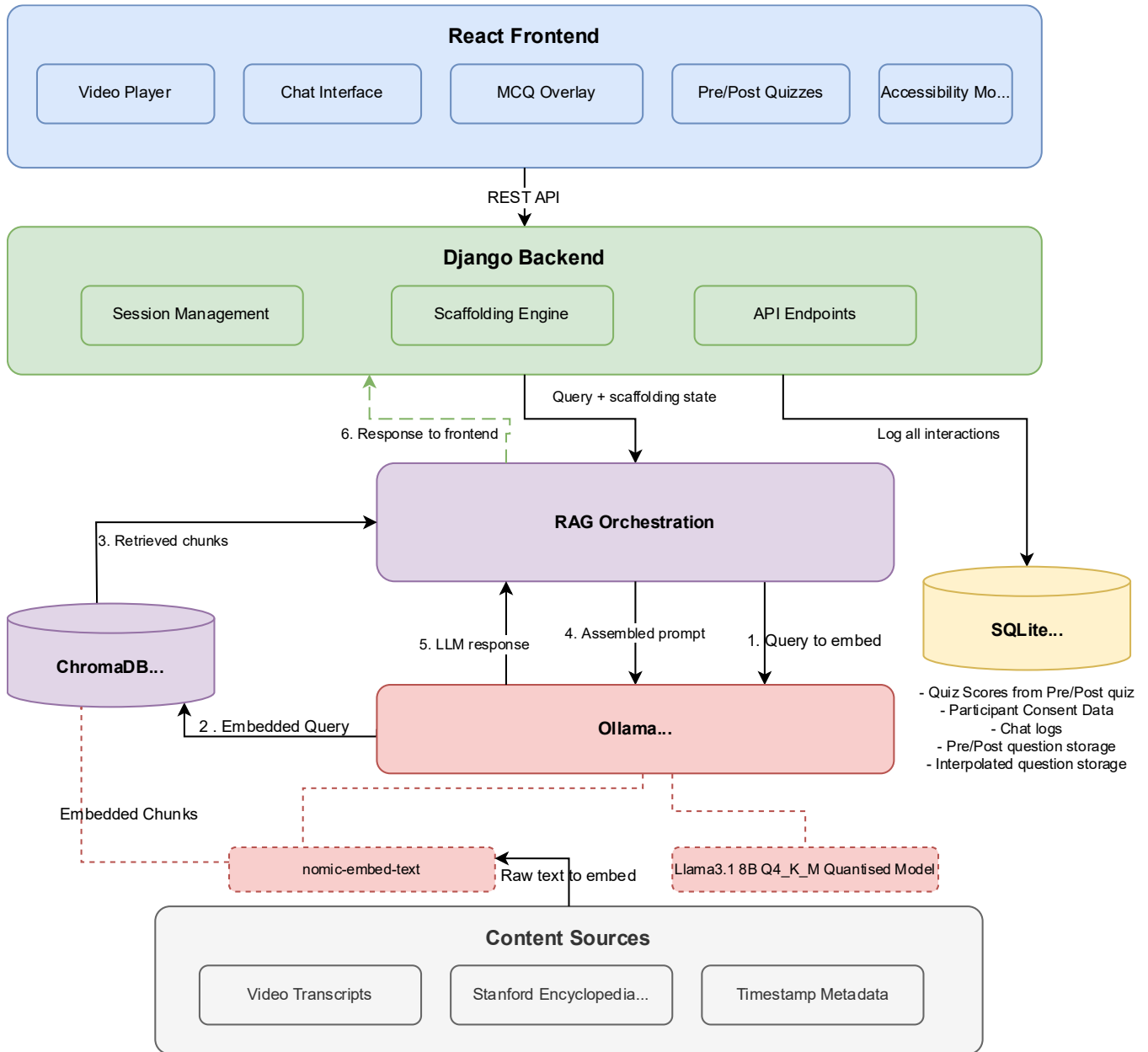


Figure 1: System Architecture

The **frontend** hosts the learner-facing surfaces: the video player, the chat interface, the MCQ overlay, the pre-quiz and post-quiz routes, and the accessibility theme controls. It also owns the live tutoring state during a session. The scaffold score, support level, the in-progress chat history and the ephemeral session identifier all live in browser memory and computed frontend-side. The trade-off this creates, particularly the loss of state on page refresh, is picked up in Section 3.5.4. A separate frontend route, */admin*, provides the teacher-facing surfaces.

The **backend** exposes REST endpoints for chat, pedagogic sub-flows including explanation evaluation, intent classification, consolidation, mastery checks, discussion checks, remediation and rewording, interpolated MCQ submission, session finalisation, and the teacher tools. The backend is largely stateless with respect to the live tutoring loop. It does not hold scaffold state, does not accumulate chat history during a session, and does not push scaffold updates back to the frontend. Persistence occurs at two defined points. Interpolated MCQ submission writes the attempt and scaffold-score snapshot. Session finalisation writes the accumulated chat history, consent record, aggregate quiz scores, and final scaffold score in one batch. This avoids per-turn database write overhead during chat, but means a session terminated before finalisation will not persist its chat history.

The **RAG orchestration layer** sits inside the backend rather than as a separate service. On each chat turn, it embeds the learner query, retrieves relevant chunks from the vector store, assembles them into a prompt with the system instructions and frontend-supplied scaffold context, and dispatches the prompt to the local model. Retrieval parameters and prompt templates are pinned to fixed values to support NFR-6.

The **model-serving layer** is a local Ollama instance running two models: *llama3.1:8b-instruct-q4_K_M* for generation and *nomic-embed-text* for embeddings. Both are served over localhost. The 8-billion-parameter model at 4-bit quantisation was chosen as a trade-off between inference latency and output quality on the available RTX 4080 Laptop GPU with 12GB VRAM.

The **storage layer** has two components. A persistent ChromaDB client holds a single collection, *tutor_corpus*. Cosine distance is set explicitly on the collection at creation time, overriding ChromaDB's L2 default. An SQLite database holds participant consent records, interpolated MCQ attempts, the question table used for pre-quiz, post-quiz, and interpolated MCQs, and per-session records containing group assignment, topic, timestamps, aggregate quiz scores, final scaffold score, and chat history written at finalisation.

The request lifecycle for a chat turn is shown numerically on the diagram. The frontend sends the learner's query, current scaffold level, and chat history to the backend. The orchestration layer then (1) sends the query to Ollama for embedding, (2) the resulting vector is sent to ChromaDB, (3) the top-matching chunks are returned to the orchestration

layer, (4) the assembled prompt, comprising system instructions, scaffold context, retrieved chunks, and the query, is sent to Ollama for generation, (5) the model's response is returned to the orchestration layer, and (6) the response, consisting of the tutor text and the citations for the chunks used, is streamed back to the frontend. Retrieved chunks themselves are not returned, and the backend does not compute or return a scaffold update; the frontend applies the scaffold-update rule locally to the resulting interaction event.

One consequence of this layout is that the backend is not a source of truth for the live tutoring loop: scaffold state lives in the browser, and chat history is batched to storage at session finalisation rather than written per turn.

3.4. Operationalising the Zone of Proximal Development (ZPD)

Vygotsky's ZPD describes the gap between what a learner can do alone and what they can do with help. Effective tutoring works inside that gap: too little support leaves the learner stuck, while too much carries them through without producing understanding. Since ZPD is not directly measurable, the system uses a tractable proxy: a scalar scaffold score updated from observable interaction events and mapped to discrete support levels. Section 3.5 specifies the update rule, thresholds, and event values.

3.4.1. Why a scalar

Compressing learner state into one scalar loses information. ZPD is sensitive to task type, the form of help offered, and the social context of interaction, none of which collapse cleanly onto one dimension. A multi-dimensional state, with separate tracks for fluency, recall, and confidence, would model the construct more faithfully, but was rejected for two reasons.

First, the system's observable events, including explanation outcomes, MCQ outcomes, hint requests, and rewatches, do not separate cleanly into independent dimensions, making extra tracks partly redundant. Second, whatever the dimensionality, the system must still choose one support level on each turn, requiring projection back to a scalar at decision time. A scalar therefore makes the support-level decision explicit, rather than requiring a more complex rule to combine separate measures of fluency, recall, and confidence.

3.4.2. Why exponential smoothing

Updating the scalar on every event in isolation would let it swing on noise: a misread question, a confidently wrong explanation, a momentary lapse. Exponential smoothing, where the new score blends the previous score with the most recent event, dampens that response. This matters because oscillation would produce a disorienting learner experience and make the support level a weaker signal of underlying competence.

Smoothing also encodes a pedagogical claim: the relevant signal is not whether the learner got the latest thing right, but how their recent trajectory compares with where they were. One limitation is that smoothing introduces no time decay. The system assumes the video lecture is completed in one sitting; a learner who pauses for ten minutes returns to the same score, on the basis that competence does not meaningfully erode within a session.

3.4.3. Why event-driven, not self-report

FR-L2 requires scaffold updates to come from observed interaction events rather than learner self-report. This follows from the Chapter 2 claim that learners are unreliable judges of their own understanding, and that chatbot interaction can encourage false fluency. Observable events also make the scaffold trajectory deterministic: the same sequence of events produces the same support-level changes.

The system does not treat all negative events equally. Hint requests and re-watches are low-cost downward signals because asking for help and re-engaging with source material are productive learning behaviours, not simple failure signals. Penalising them heavily would discourage the behaviours a tutoring system should make safe.

3.5. Scaffolding engine design

The scaffolding engine implements the ZPD proxy at runtime. It maintains a scalar scaffold score for the current session, updates it from interaction events, and maps it to one of five support levels on each turn. The session-level flow in which it operates, including the pre-quiz, video loop, interpolated MCQs, mode selection and post-quiz, is shown in Appendix 8.

3.5.1. Five-level support model

The engine selects from five support levels, ordered from most to least supportive:

- **L1, Full Support.** The tutor models or directly explains the concept under discussion.
- **L2, High Support.** The tutor explains briefly and confirms understanding.
- **L3, Moderate Support.** The tutor cues or hints towards the answer rather than supplying it.
- **L4, Low Support.** The tutor prompts the learner to reason, offering minimal scaffolding.
- **L5, Minimal Support.** The learner reasons independently; the tutor’s role is to acknowledge and respond rather than guide.

The level selected on each turn is determined by the current scaffold score against four thresholds displayed in the table below.

Score	Level
< -0.5	1
< 0.5	2
< 1.5	3
< 2.5	4
>= 2.5	5

Table 4: Scaffold score level thresholds

Five levels were chosen because three were too coarse to capture meaningful support changes, while seven would require distinctions the prompt-engineered tutor could not reliably deliver. Five also maps onto a familiar pedagogical progression: full instruction, guided instruction, prompted reasoning, cued reasoning, and independent reasoning.

The thresholds are asymmetric. The L1/L2 boundary sits at -0.5 and the L4/L5 boundary at 2.5, so the score has to travel further to reach the minimal-support state than to reach the full-support state. This reflects caution about removing support: a learner who has performed well briefly should not immediately be left to reason alone, but a learner who has struggled should receive additional support without delay.

3.5.2. Score update rule and event weighting

On each scaffold-relevant event, the score is updated by:

$$S_{new} = 0.5 \times S_{old} + 0.5 \times E_{recent}$$

The previous score and most recent event are weighted equally to keep the engine responsive without making it volatile.

Explanation evaluation applies when the learner attempts to explain a concept after a correct interpolated MCQ:

Outcome	Value
Correct	+2.5
Partial	+1
Incorrect	-1

Table 5: Score update rule after correct interpolated MCQ

MCQ-flavoured events vary by context:

MCQ context	Correct	Incorrect
Initial interpolated MCQ	(no direct update)	(no direct update)
Discussion check	+0.5	-0.5
Mastery check	0	-0.5
Consolidation	0	0
Remediation, easy	+1	(no direct update)
Remediation, post-intervention	+0.5	-1

Table 6: Different MCQ events with update rules

Initial interpolated MCQs do not update the score directly: correct answers route to explanation evaluation, while incorrect answers route to remediation. Consolidation MCQs are neutral because they support low-stakes review rather than assessment.

Other events:

- Hint request: -0.3
- Rewatch: 0 (neutral)
- Successful reword of a question: +0.5
- Remediation backend returns no easy question: -1
- Remediation generation request fails: -2

Hint requests carry only a small cost because asking for help is a productive learning behaviour. Rewatching is neutral for the same reason: re-engaging with source material should not be penalised. The remediation-failure values reflect system errors rather than learner performance, but still lower the score because the learner experiences a degraded interaction and therefore receives more support.

One implementation issue affected explanation evaluation. The function contains a first-attempt branch (+4 correct, +1.5 partial, -0.5 incorrect), but this branch is not reached at runtime because the attempt counter is incremented before the function call. The deployed system therefore applies the later-attempt weights (+2.5 / +1 / -1) to all explanation evaluations. This was identified after data collection had closed, so the evaluated system is reported with those uniform later-attempt weights.

The score is not clamped. Correct explanations can push it above 2.5 and repeated incorrect attempts can push it below -0.5, although the support level remains bounded at L5 and L1. This preserves the smoothing rule: clamping would make boundary scores disproportionately sensitive to recent events, weakening the intended history-recency blend.

3.5.3. Initial level from the pre-quiz

The scaffold score for a tutoring session is initialised from the learner's score on the topic-specific pre-quiz subset:

$$S_0 = (\text{correct} \div \text{total}) \times 6 - 2$$

This maps a perfect pre-quiz score to +4, corresponding to L5, and a zero score to -2, corresponding to L1. Intermediate scores fall across the threshold band. The mapping gives learners an initial support level based on prior topic knowledge: learners who already know the material begin with minimal support, while learners with no demonstrated prior knowledge begin with full support.

The pre-quiz subset is topic-specific: items for Topic A contribute only to Topic A's initial scaffold score. An earlier version used the whole-quiz total rather than the topic-specific subset, but this was caught during development testing and fixed before participant data collection.

3.5.4. State management and limitation

The live scaffold state is held in browser memory by the React controller hook that manages the tutoring loop. The backend persists snapshots after each interpolated MCQ attempt and at session finalisation, but does not hold or compute the live score between those points. This keeps the chat and MCQ loops fast and leaves the backend stateless with respect to scaffold logic, but scaffold state is lost on page

refresh or browser crash. In the within-subjects study, where sessions ran in one sitting on a stable machine, this limitation did not manifest. A deployed system used in less controlled conditions would need persistent server-side scaffold state.

3.6. Retrieval-augmented generation pipeline

The tutor’s chat replies are grounded in a small, course-specific corpus rather than relying on the base model’s parametric knowledge. This supports NFR-2: the model should answer from material presented or selected for the course, not from pre-training alone. This section describes the corpus, chunking strategies, embedding and retrieval steps, and how retrieved chunks are passed to the model.

3.6.1. Corpus and ingestion

The corpus contains five Stanford Encyclopedia of Philosophy entries (liberalism, mill-moral-political, mill, democracy, and rousseau) and two Wireless Philosophy lecture transcripts: liberalism and representative-democracy. The SEP sources were fetched from their public entry URLs on 16 April 2026, with URL, title, local filename, and access date recorded in the ingestion manifest shown in Appendix 8.11. The transcript files were stored as .vtt files derived from the YouTube caption tracks for Geoff Pynn’s Wireless Philosophy videos Liberalism and Representative Democracy (Wireless Philosophy, 2023a/b). The SEP entries provide deeper philosophical definitions and background material, while the transcripts ground the tutor in the specific videos watched by participants.

The video transcripts were obtained from the YouTube caption tracks and stored locally for ingestion. SEP entries were fetched as HTML by URL and parsed with BeautifulSoup, which removes sidebars, bibliography, footer, and scripts. A short blocklist removes one further section, “a note on texts and references”, because it contains editorial metadata rather than learning content. The live system contains 269 chunks: 250 from SEP and 19 from transcripts.

The two source types are chunked differently because they have different structures. SEP articles are chunked along the h2/h3 heading hierarchy. Where a section exceeds a 500-token cap, the splitter packs paragraphs greedily until the cap is reached. This preserves the encyclopaedia’s own organisation, so most chunks correspond to a coherent argumentative move rather than arbitrary windows.

Transcripts are chunked by time, with a 40-second soft target and a 60-second hard cap. The chunker respects caption cue boundaries, and after the 40-second threshold, continues until it reaches a sentence-final character, forcing a cut only at 60 seconds. Adjacent chunks overlap by 15 seconds. A tail-merge rule folds the final chunk into its predecessor if it is shorter than 15 seconds or contains fewer than 40 words. Sentence-boundary preference reduces the chance of chunks beginning or ending mid-clause, preserving semantic coherence.

3.6.2. Embedding and storage

Both ingestion and queries use `nomic-embed-text`, served locally via Ollama. Ollama lists `nomic-embed-text` as an embedding-only model, while Nomic’s documentation identifies task-specific prefixes for retrieval: `search_document:` for document chunks and `search_query:` for queries (Ollama, no date; Nomic, no date). These prefixes are applied at ingestion and query time respectively.

Vectors are stored in ChromaDB with cosine distance rather than ChromaDB’s default L2 distance. Fixing the metric at collection creation keeps retrieval distances stable for the tuned cutoff. ChromaDB’s documentation identifies L2 as the default distance function, with `cosine` and `ip` available as configurable alternatives (ChromaDB, no date).

Queries are lowercased before embedding; documents are not. This asymmetry was added after a development-testing observation: “What is the Harm Principle?” returned unrelated Rousseau biographical content at cosine distance 0.34, while “what is the harm principle” returned the correct content at 0.17. This suggested that query capitalisation affected retrieval behaviour under the chosen embedding model and pipeline. Other query normalisation steps, including whitespace and trailing-punctuation handling, are described in the implementation chapter.

3.6.3. Retrieval and prompt assembly

For each tutor turn, the chat service embeds the normalised query, asks ChromaDB for the top three nearest chunks, and filters out chunks with cosine distance above 0.65. A top-k value of 3 keeps the prompt short enough for scaffold context and conversation history to remain salient.

The 0.65 cutoff is deliberately permissive. Development-testing notes recorded relevant-match distances of 0.15-0.30 after the lowercasing fix, but unrelated-chunk distances under the final query pipeline were not systematically measured. Without a measured upper bound for relevant matches across plausible student phrasings, a stricter cutoff risked excluding legitimate retrievals. The cutoff therefore favours retrieving marginal chunks and relying on the fallback path and model grounding behaviour, rather than rejecting potentially useful context.

Two fallback behaviours handle weak retrieval inputs. If the student message is very short or low-information, such as “idk”, retrieval uses the topic name combined with the interpolated question to avoid a near-empty embedding. If no chunk passes the 0.65 cutoff, the system returns a non-grounded reply with an empty citations list rather than refusing to answer. This reduces refusals on legitimate phrasings, but means some replies can escape grounding. NFR-2 faithfulness measurement covers the grounded path; the fallback rate is reported in the evaluation chapter.

The model receives separate system and user prompts. The user prompt is assembled in a fixed order: session metadata, the interpolated question and any post-answer context, a "Retrieved learning context:" block listing the surviving chunks as [1], [2], [3], a "Conversation so far:" block and a "Student message:" block. Numbered chunk markers let the model produce inline citations the response validator can check.

The citation payload returned to the frontend lists, per cited chunk, source type, source identifier, title, URL, a human-readable location, and (for transcripts) a numeric start_time in seconds. SEP locations take the form "Section > Subsection". Transcripts use m:ss–m:ss, with start_time exposed separately so learners can jump directly to the cited moment in the video.

3.7. Guardrail design: chat channel

The chat channel is the system’s main attack surface. A student can try to extract the correct MCQ answer directly, for example by asking “which is right?”, or indirectly through transformed disclosures such as translation, encoding, sentence completion, hypothetical function output, role-play, claimed authority, or “ignore previous instructions” prompts.

Since the system uses a locally deployed open-weights model, it has no provider-side moderation layer. A reliable runtime leak detector was also outside the project scope: a learned classifier would require labelled adversarial data, while a rule-based detector would risk over-refusal or easy bypass through paraphrase. The design is therefore prompt-led.

An answer-protection block is appended to every chat system prompt. It instructs the model to provide scaffolded conceptual help without naming an option letter, identifying a position such as “the third option”, restating an option’s wording, confirming or denying a named option, or eliminating options until one remains. It also blocks transformed disclosure requests and common jailbreak framings, while instructing the model to respond normally rather than lecture about the rules. The blocked transformation list was built iteratively through adversarial testing, as described in Section 3.2.3.

Runtime checks for retrieval bounds and citation validity protect grounding and output format, not answer protection. Answer protection is therefore deliberately concentrated in the prompt-level guardrail rather than distributed across a separate classifier or post-generation filter. This keeps the design lightweight and feasible for the project scope, but leaves leaked-answer detection as an explicit limitation discussed in Section 3.12.

3.8. Teacher-facing design: reducing friction for adoption

3.8.1. Motivation and scope

A tutoring system that requires custom question authoring for every video is unlikely to be adopted, regardless of its pedagogical merits. Teacher time is the limiting resource: writing interpolated questions, checking tutor behaviour, and inspecting retrieval quality are the real deployment costs. The teacher-facing design therefore reduces this cost by exposing internal system state through the */admin* interface and drafting candidate questions with grounding controls.

The scope is narrow. This is not a full content-management system, but a small set of tools for a single instructor running a course: question generation and editing, participant inspection, corpus management, and retrieval debugging.

3.8.2. The */admin* interface: Questions, Participants, RAG debug

The /admin interface exposes three tabs. Questions lists generated questions by topic and quiz type, supports edit/delete operations, and can trigger generation. Participants lists active sessions with study summary metrics, CSV export, and a chat-history viewer; it is not a learning analytics dashboard. RAG debug provides corpus overview, source management, and ad-hoc retrieval testing with returned distances. Retrieval testing is included so teachers can diagnose off-topic chunks, missing content, or ingestion errors without reading code.

3.8.3. Assistive question generation: layered grounding controls

Question generation operates over one transcript excerpt at a time, drawn from evenly spaced chunks of the topic transcript. Six grounding controls apply.

First, the prompt restricts the question to the excerpt and forbids outside concepts.

Second, excerpt selection skips chunks already used by saved questions.

Third, the model must emit a verbatim support_quote, and the backend rejects drafts whose quote, after whitespace normalisation, is not a substring of the excerpt.

Fourth, duplicate question texts and reused excerpts are filtered before persistence.

Fifth, a second-model reviewer checks whether the question is answerable from the excerpt alone, whether exactly one option is supported, whether distractors are paraphrased rather than quoted, and whether attribution is correct.

Finally, persistence is atomic: if the requested batch cannot be assembled within the retry budget, nothing is saved.

These layers correspond to the failure modes discussed in the next subsection.

3.8.4. Observed failure modes and the case for human review

Development testing surfaced several failure modes that shaped the grounding controls in Section 3.8.3. Malformed JSON led to the parse-retry loop. Hallucinated or paraphrased grounding quotes led to the verbatim-substring check. Repeated drafting against the same excerpt led to duplicate suppression. More subtle failures, including ambiguous distractors, wrong-person attribution, inverted stem-answer logic, and questions drawn from rhetorical or open-ended passages, motivated the second-model reviewer.

The reviewer reduces these risks but does not remove the need for human judgement. It fails open if its own LLM call errors, meaning a question can pass review by accident. It also cannot reliably detect every pedagogical weakness: a question may be grounded, non-duplicate, and reviewer-approved while still assessing a peripheral sentence rather than the intended concept.

The current system therefore supports teacher correction after generation rather than enforcing human approval before publication. Generated questions persist directly to the table learners read from, although teachers can edit or delete them through the admin interface. This is a current-system constraint rather than a complete human-in-the-loop review design, and is identified as a limitation in Section 3.12.

3.8.5. Authorisation: a deferred concern

The /admin interface is unauthenticated in the deployed system. This is acceptable for the current scope: deployment is local, single-instructor, and limited to a supervised user study. Wider deployment would require, at minimum, authenticated admin access and authorisation checks on question modification, participant data, and RAG-management endpoints. Multi-tenant, institutional, or unsupervised access would require a different threat model, so authentication was scoped out and is acknowledged as a generalisability constraint in Section 3.12.

3.9. Question store and reuse strategy

Generated and authored questions persist in a single SQLite table, keyed by topic and quiz_type (pre-quiz, interpolated, or closing). All participants on a given topic draw from the same pool. The pre-quiz and post-quiz use the same stored questions, with post-quiz order shuffled client-side.

This supports the within-subjects design for RQ1: identical pre/post items remove item difficulty as a source of variance between measurements. Participants do not see correct answers after the pre-quiz, and the only intervention between pre and post is the lecture itself, so improvement is attributed to learning from the video rather than exposure to a marked answer key.

Interpolated questions are drawn from a separate pool and ordered by video timestamp, so participants watching the same lecture encounter the same question at the same point.

Questions can be edited or deleted through the admin interface, but there is no per-participant copy-on-write or randomisation layer.

3.10. Learner-facing user interface

3.10.1. Layout and the interpolated question overlay

The learner-facing interface is built around a session page with a video player on the left, tutor chat on the right, and pre/post-quiz routes at the start and end. Its main design feature is the interpolated question overlay. When the video reaches a configured timestamp, playback pauses and an MCQ appears over the video frame rather than on a separate page.

The overlay dismisses once the question is resolved, either through a correct answer or exhaustion of attempts. The question text is also mirrored into the chat panel, allowing the learner to refer back to it or ask the tutor about it after playback resumes. This separates the overlay's role as a transient lecture checkpoint from the chat panel's role as the persistent tutor-interaction surface. Figure 2 shows the design wireframe below.

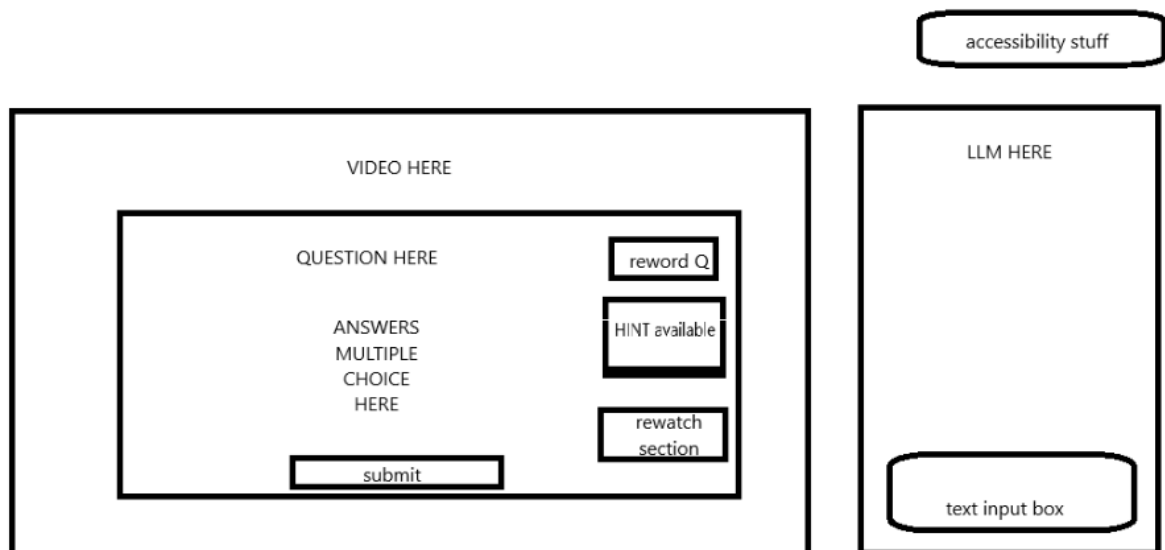


Figure 2: Initial wireframe

3.10.2. Accessibility (with known limitations)

Accessibility work currently consists of initial provisions rather than a full compliance implementation. Both learner and admin interfaces provide light, dark,

high-contrast, and colourblind display modes, with most core UI colours driven by shared theme tokens. Main learner views also use semantic landmarks and ARIA labelling, though some custom controls still need fuller keyboard and assistive-technology support.

The dark theme was used during development and the participant study. Alternative learner-interface variants were not systematically tested. One visible issue was found in the colourblind variant: a white-on-yellow chat bubble had poor contrast and would need a theme-token adjustment. Full accessibility compliance is not claimed; a systematic audit is identified as future work in Section 3.12.

3.11. Development tools

The project was developed in Visual Studio Code, with Git version control against a private GitHub remote; the submission package is not delivered as a Git repository. The Python backend uses `uv` for dependency management and command execution via `pyproject.toml`. The frontend uses `npm`, `Vite`, TypeScript checking (`tsc -b`), and `ESLint`. Frontend tests use `Vitest` with `jsdom`, `React Testing Library`, and `jest-dom`; backend tests use Django’s test runner, `unittest`, and Django REST Framework’s `APITestCase`. Diagrams, including the Appendix 8.2 session-flow diagram, were produced with `draw.io`. OpenAI Codex supported code debugging during development.

3.12. System limitations

The limitations below concern constraints in the current system design described in this chapter. Implementation-testing limitations are discussed in Section 4.4, and evaluation limitations are discussed in Section 5.7.

The chat channel has no post-generation classifier for leaked MCQ answer letters. Answer protection rests on the system-prompt block, so runtime leak detection remains a clear extension.

The retrieval cutoff is permissive rather than fully empirically tuned. Relevant-match distances were recorded in the 0.15–0.30 range, but unrelated-chunk distances under the final query pipeline were not systematically measured. The fallback path also allows some replies to escape grounding when no chunk passes the cutoff.

Live scaffold state is held in browser memory. This keeps the tutoring loop responsive, but scaffold state does not survive page refreshes or browser crashes.

Pre-authored generated questions persist directly to the table learners read from, with no draft or approval state. Teachers can edit or delete after generation, but publication is not gated on review.

The grounding-reviewer pass for generated questions fails open on exception: if the reviewer's own LLM call errors, the question passes review by default.

The `/admin` interface is unauthenticated. This is acceptable for the local, supervised deployment, but wider use would require authenticated admin access and endpoint-level authorisation.

Learner-interface accessibility remains partial. Both learner and admin interfaces provide display modes, and main learner views use semantic landmarks and ARIA labelling, but some custom controls still need fuller keyboard and assistive-technology support. Alternative theme variants were not systematically tested. One known issue is the colourblind-mode white-on-yellow chat bubble, which has poor contrast and would require a theme-token fix.

4. Implementation

4.1. Project Setup and build

The codebase is organised around two main sibling directories at the repository root: `backend/`, a Django project, and `tutor-app/`, a React application. The Django backend uses a single `tutor` app, with functionality split across modules for ingestion, retrieval, chat orchestration, question generation, REST views, and data models. Backend dependencies are managed with `uv` via `pyproject.toml` and `uv.lock`. Runtime dependencies include Django, Django REST Framework, `django-cors-headers`, ChromaDB, BeautifulSoup, `webvtt-py`, `tiktoken`, and `requests`; `ragas`, `litellm`, `pandas`, and `Pillow` are scoped to an optional evaluation extra, allowing participant-facing deployments to avoid installing the evaluation stack.

The frontend uses Vite, TypeScript, ESLint, and npm. Its build script, `tsc -b && vite build`, runs type checking before bundling, so type errors fail the build. Figure 3 shows the welcome surface used at session start.

The SQLite data model is small and study-focused. It stores participant consent and session metadata, generated and stored MCQs, interpolated MCQ attempts, aggregate pre- and post-quiz scores, per-attempt `scaffold_score_after` values, a final scaffold score for each completed session, and chat histories persisted when sessions are finalised. This supports reproducible evaluation and CSV export rather than a broader participant account-management mode

RAG-Grounded LLM Tutoring System

Admin →

PARTICIPANT ID
93410

Eligibility & consent

Please read and confirm each of the following before proceeding.

- ☐ I have read and understood the [participant information sheet](#).
- ☐ I am over the age of 18.
- ☐ I am a university student studying a STEM subject (e.g. Computer Science, Engineering, Mathematics, or Physics).
- ☐ I have read and understood the [consent form](#) and agree to participate in this experiment.
- ☐ I understand that there is an optional post-session questionnaire I will be invited to complete after the study, and that my responses will be anonymous and linked only by my participant ID.

Session settings

Participant type

☒ **Group A**
Liberalism (AI) -> Rep. Democracy (video-only)

☐ **Group B**
Liberalism (video-only) -> Rep. Democracy (AI)

☐ **Group C**
Rep. Democracy (AI) -> Liberalism (video-only)

☐ **Group D**
Rep. Democracy (video-only) -> Liberalism (AI)

Downloaded local model

llama3.1:8b-instruct-q4_K_M

Selected model: llama3.1:8b-instruct-q4_K_M

Start session

Figure 3: Welcome surface at session start, showing participant ID, group setup selection, model selection and consent checkboxes.

During development, the backend and frontend run as separate localhost processes, with Ollama as a third process serving the generation and embedding models. There is no orchestration layer that starts them together; Appendix 8.4 documents the startup sequence.

4.2. Streaming chat path

The chat channel uses two endpoints rather than one handler with a streaming flag: `POST /api/chat` returns buffered JSON, while `POST /api/chat/stream` returns a `StreamingHttpResponse` as `application/x-ndjson`. Both share retrieval, prompt assembly, and grounded/fallback routing, but differ in citation handling.

The buffered path validates the full response before returning it. Grounded responses must contain at least one valid citation marker in the range `[1]` to `[max_index]`. If validation fails, generation is retried once with an added grounding-retry prompt. If both grounded attempts fail, the orchestrator falls back to a non-grounded response, validated to contain no citation markers. Because validation, retry, and fallback all occur before return, the buffered endpoint never sends a malformed result. The buffered handler, the two retry helpers, and the retry prompts are given in Appendix 8.10.

The streaming path cannot validate before sending, because malformed text may already have reached the client. It therefore streams model chunks as `delta` events, accumulates the full response, then repairs citations before sending the `final` event. On the grounded branch, repair removes citation markers outside the valid range; if no valid marker remains, `[1]` is injected at the first sentence boundary. On the fallback branch, all citation markers are removed. If repair changes the text, a `replace` event is sent before the `final` event so the client can overwrite its displayed buffer.

The injected `[1]` is a display-level repair, not a substantive grounding correction. It keeps the rendered response well-formed in cases where the buffered path would have retried. Figure 4 shows a streamed tutor reply with inline citation markers and the citation panel beneath. The streaming handler, streaming loop, and repair functions are given in Appendix 8.9.

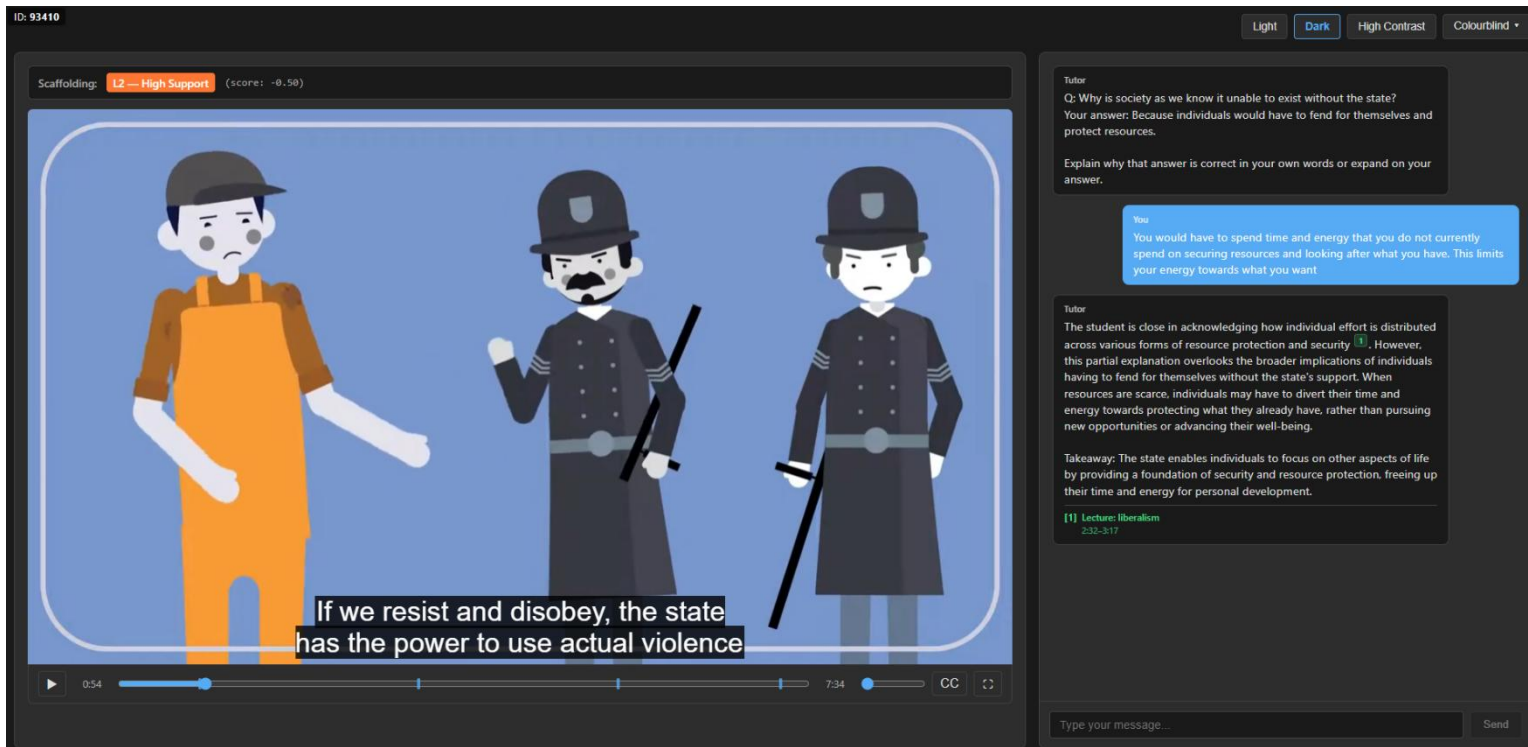


Figure 4: Streamed tutor reply with the inline citation marker [1] resolving to a transcript chunk shown in the citation block beneath

Fallback behaviour is harder to debug because fallback responses are represented only by an empty citations array, rather than a dedicated response field.

4.3. Question generation

4.3.1. Pre-authored generation

The pre-authored pipeline is exposed through `POST /api/questions/generate`, handling both pre/post and interpolated quiz types. Each candidate excerpt receives up to four model attempts: the initial attempt plus three grounding retries. Within each attempt, validation runs in sequence: JSON parsing, schema checking, support_quote substring containment, duplicate suppression, and grounding review. The full per-excerpt loop is given in Appendix 8.8, and the generation prompt in Appendix 8.6.

The substring check normalises both the generated quote and source excerpt by collapsing whitespace and stripping matching outer quote characters before testing containment. The reviewer returns a YES/NO judgement; any response not starting

with YES is treated as rejection, while reviewer-call exceptions are caught and treated as acceptance. The reviewer prompt and checklist are given in Appendix 8.7.

If an excerpt exhausts its attempts without producing an accepted question, the handler pulls the next index from a fallback pool of unselected excerpts and adds it to the queue. Accepted questions are accumulated in memory and committed only once the requested count is reached. If the loop terminates short, nothing is written and the response returns 502 with a per-attempt error summary.

4.3.2. In-chat MCQ generation

The four live sub-flows (consolidation, mastery, discussion-check, and remediation) are exposed as separate endpoints returning buffered JSON rather than streaming over the NDJSON channel. The frontend renders each result as a structured MCQ card in the chat panel, so the chat surface mixes streamed tutor prose with non-streamed MCQ payloads. Figure 5 shows a remediation MCQ card after a failed interpolated question.

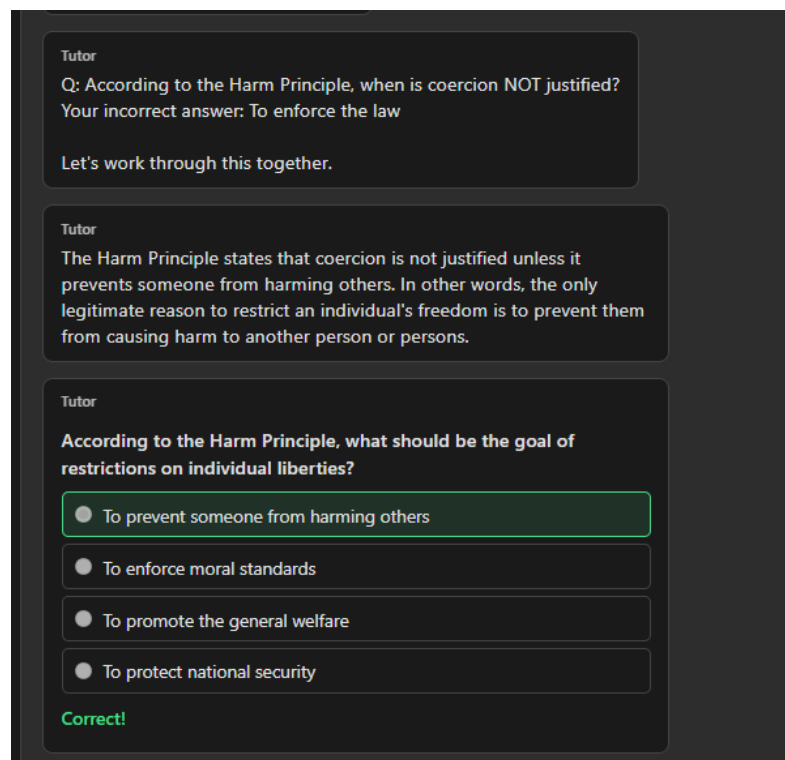


Figure 5: Remediation MCQ card rendered in the chat panel.

All four sub-flows ground generation in retrieved RAG context rather than conversation history or the originating interpolated excerpt. The handler re-queries

ChromaDB using the original question text and topic, then inserts the surviving chunks into the prompt alongside the sub-flow input: the learner's explanation, learner's question, or original correct answer. Each prompt requests a flow-specific JSON shape, and all results pass through a shared structural validator requiring question text, exactly four options, and `correct_index` in the range 0–3.

Three of the four sub-flows also enforce citation-marker presence. Remediation's easy question bypasses this by design, calling the validator with `citations=None`. Unlike the pre-authored pipeline, live MCQs have no `support_quote`, substring-containment check, second-model grounding review, or persistence. They are returned as dataclass payloads, rendered once, and discarded when the chat session ends. Remediation also generates from scratch rather than fetching a pre-authored easy question.

The lighter validation surface is a deliberate trade against latency: the sub-flows fire mid-session and must return within a chat-turn budget, so the per-question reviewer pass and atomic-batch machinery used by the pre-authored pipeline would be the wrong cost profile for content that is rendered once and discarded.

4.4. Testing

The test suite is split across backend and frontend. Backend tests live in `backend/tutor/tests.py` and run under Django's test runner using `unittest` and DRF's `APITestCase`. Frontend tests live alongside the components and hooks they exercise under `tutor-app/src/`, and run under Vitest with `jsdom`, React Testing Library, and `jest-dom`.

No test calls a live Ollama or ChromaDB instance. Model generation is patched at the `generate_text` boundary using prepared mock responses, while retrieval is patched at the chunk-selection helpers so tests inject context directly. This limits the suite to behaviour above the generation and retrieval boundaries, but still exercises the application code paths that orchestrate prompts, validation, routing, and response handling during participant sessions.

Backend coverage includes endpoint validation for chat, model listing, in-chat sub-flows, pre-authored question generation, participant session lifecycle, and CSV export. Service-

layer tests cover prompt assembly, citation validation, retry behaviour, addendum precedence, parser ambiguity rejection, and grounded/fallback handling. Frontend coverage targets controller routing in `useTutorController`, consolidation/remediation fallback branches in `useConsolidation`, and component behaviour for the welcome flow, quiz scoring, and in-video question overlay.

Several behaviours are not directly tested: the streaming chat endpoint, post-hoc citation repair, frontend NDJSON consumer, `support_quote` rejection path, grounding-reviewer fail-open path, and scaffold-score update rule. There are no end-to-end tests covering the full frontend-to-backend-to-retrieval flow.

5. Evaluation

This chapter evaluates whether the implemented system met the project aim: developing a locally deployable, RAG-grounded LLM tutoring system for video-based learning, designed to support learners through adaptive scaffolding and improve immediate retention compared with passive video viewing. Evaluation is organised around the three research questions, the quantitative thresholds attached to them, and the supporting objectives set out in the feasibility study.

A working system was deployed on consumer hardware, and a within-subjects retention study was completed with six participants. The following sections report the retention results, RAGAS faithfulness and correctness scores, guardrail performance, and objective-level evaluation.

5.1. Evaluation Criteria

This system is evaluated against three research questions, each linked to a quantitative test or threshold:

- **RQ1, retention.** Does the RAG-grounded adaptive tutor improve immediate retention compared with video-only learning? This is evaluated through a within-subjects experiment comparing post-quiz improvement in the AI-tutor and video-only conditions, using a one-sided paired Wilcoxon signed-rank test (Wilcoxon, 1945).
- **RQ2, faithfulness and correctness.** Does the tutor maintain grounded responses using retrieved content? Faithfulness is measured with RAGAS over a fixed evaluation set, with a target mean score above 0.8. Correctness is reported as a secondary NFR-2 criterion over the same set, with a target above 0.9.
- **RQ3, guardrails.** How effective are the system’s guardrails against direct answer extraction? This is evaluated as the deflection rate against a predefined adversarial prompt set, with a target above 90%.

Two boundaries apply to these claims. First, retention is measured immediately after the video, so no delayed-retention claim is made. Second, guardrail performance is tested against a predefined adversarial set drawn from published taxonomies, not against novel or adaptively generated attacks.

These thresholds were selected as pragmatic success criteria rather than claims of production readiness. A faithfulness target above 0.8 was used to require most response claims to be attributable to retrieved context while allowing limited pedagogical paraphrasing. Correctness was set higher, at 0.9, because an educational tutor can only be useful if its answers are usually conceptually accurate. The 90% guardrail target was chosen as a demanding but realistic benchmark for a prototype system tested against predefined adversarial prompts; it indicates strong resistance to direct answer extraction without claiming complete security against adaptive attack.

5.2. Aims and objectives review

The project aimed to develop and evaluate a locally deployable, RAG-grounded LLM tutoring system for video-based learning, using adaptive scaffolding to support learners and measuring immediate retention against passive video viewing. The system aim was achieved operationally: the application ran on consumer hardware, the within-subjects study was completed with six participants, and all three research questions were evaluated quantitatively. However, the RAGAS faithfulness threshold was not met, so the grounding objective was only partially achieved.

5.2.1. Objective 1: Literature review

Chapter 2 reviewed ZPD-based scaffolding, pedagogical risks of generative AI in education, AI-assisted video learning, local LLM deployment, and retrieval-augmented generation. It identified the project gap: no reviewed system combined video-grounded RAG, ZPD-calibrated scaffolding, local deployment, and human evaluation in a single artefact.

5.2.2. Objective 2: System design

Chapter 3 specified a five-layer architecture combining local inference, RAG over video transcripts and supplementary material, interpolated knowledge checks, adaptive scaffolding, and prompt-level answer-extraction guardrails. The scaffolding engine, RAG pipeline, and guardrail design were specified sufficiently to support implementation.

5.2.3. Objective 3: System Implementation

The system was implemented end-to-end on a consumer laptop with 12 GB dedicated GPU memory, 32GB system RAM, using Ollama with a quantised Llama 3.1 8B model for local inference. Implementation covered the streaming chat path, four in-chat MCQ sub-flows, pre-authored question generation with reviewer validation, and the participant-facing session lifecycle.

5.2.4. Objective 4: Evaluating Grounding

Faithfulness was evaluated with RAGAS across three independent runs of a 47-item evaluation set. Run means were 0.7416, 0.7401, and 0.7603, giving a mean of run-means of approximately 0.747 against the 0.8 target. The target was missed in all three runs, with low variance across runs, indicating a consistent shortfall rather than measurement noise. Correctness, the secondary NFR-2 measure, passed with a mean pass rate of 95.7% against the 0.9 target.

5.2.5. Objective 5: Adversarial guardrail testing

A tiered adversarial harness was constructed across three prompt sets covering eighteen attack categories adapted from Liu et al.'s jailbreak prompt-pattern taxonomy, Wei et al.'s safety-training failure modes, and Schulhoff et al.'s prompt-hacking taxonomy (Liu et al., 2023; Wei et al., 2023; Schulhoff et al., 2024). Combined deflection reached 92.6%, exceeding the 90% target. Section 5.5 discusses per-category results and failures.

5.2.6. Objective 6: Within-subjects retention experiment

A partially counterbalanced within-subjects experiment was conducted with six participants. Each completed both AI-tutor and video-only conditions across two philosophy topics, with topic and condition order varied across four counterbalancing groups. All participants completed both sessions and the associated pre/post quizzes.

5.2.7. Objective 7: Combined analysis of experiment and interaction-log data

Retention scores were analysed alongside chat-message counts, MCQ attempts, and post-session questionnaire responses. This allowed quantitative outcomes to be interpreted alongside learner interaction and feedback data. The clearest pedagogical

finding came from the free-text responses: despite favourable Likert ratings, participants wanted clearer feedback on whether their answers were correct. Section 5.6 discusses this finding.

5.3. RQ1 Retention

5.3.1. Experimental design

Each participant completed two sessions: one in the AI-tutor condition and one in the video-only condition, across two philosophy topics, Liberalism and Representative Democracy. Topic order and condition order were counterbalanced across four groups.

Group	First session	Second Session
A	Liberalism (AI)	Representative Democracy (video-only)
B	Liberalism (video-only)	Representative Democracy (AI)
C	Representative Democracy (AI)	Liberalism (video-only)
D	Representative Democracy (video-only)	Liberalism (AI)

Table 7: Counterbalancing groups for the within-subjects retention experiment.

Six participants were distributed unevenly across the four groups: A: 2, B: 1, C: 2, D: 1. This is discussed as a limitation in Section 5.7. Both topics were represented equally in the AI condition, with three participants using the tutor on Liberalism and three on Representative Democracy. In each session, participants completed a pre-quiz, watched the video, and completed a post-quiz drawn from the same seven-item pool. The dependent variable is the within-participant difference between AI-tutor gain and video-only gain, measured in percentage points.

5.3.2. Results

All six participants completed both sessions.

Participant ID	Group	AI topic	Video gain (%)	AI gain (%)	Diff	Chat msgs
76878	C	Rep. Dem.	+0	+14	+14	33
28996	A	Liberalism	-14	+43	+57	30
33383	D	Liberalism	+14	+43	+29	35
70982	A	Liberalism	+14	+43	+29	32
29917	C	Rep. Dem.	+43	+57	+14	34

33799	B	Rep. Dem.	+14	+29	+15	47
-------	---	-----------	-----	-----	-----	----

Table 8: Per-participant retention results across AI-tutor and video-only conditions.

Mean video-only gain was 11.83 percentage points, while mean AI-tutor gain was 38.17. The mean within-participant difference was 26.33 percentage points with a median difference of 22. A one-sided paired Wilcoxon signed-rank test gave $W = 21$, $p = 0.0156$; the two-sided test gave $W = 0$, $p = 0.0312$. All six participants showed greater gain in the AI-tutor condition.

5.3.3. Interpretation

RQ1 is supported. The AI-tutor condition produced a larger immediate retention gain for every participant, with a mean advantage of 26.33 percentage points. At $N = 6$, $p = 0.0156$ is the strongest possible one-sided Wilcoxon outcome, reached only when all six within-participant differences point in the same direction. This supports the claim that every participant in this sample improved more after tutor-supported study than after passive video viewing, but does not justify a precise population-level effect-size estimate.

Two patterns qualify the result. Participant 28996 scored lower after the video-only condition, producing the largest within-participant difference (+57). The median difference of 22 percentage points is therefore the more robust summary, and removing this participant would still leave a mean advantage of roughly 20 percentage points with all remaining differences favouring the tutor. Chat-message counts were also tightly clustered, between 30 and 47, suggesting the result was not driven by one or two unusually heavy tutor users.

5.3.4. Limitations specific to RQ1

The retention measure is immediate, so no delayed-retention claim is made. The seven-item quiz changes in coarse increments of approximately 14 percentage points per item, limiting measurement resolution. Counterbalancing was incomplete because six participants cannot be evenly distributed across four groups.

5.4. RQ2: Faithfulness and correctness

5.4.1. Evaluation method

Faithfulness and correctness were evaluated using RAGAS against a held-out 47-item question set spanning both module topics, with a small number of out-of-scope philosophy questions included to test redirection behaviour. Each question was issued to the deployed system, then the retrieved chunks and generated response were captured for scoring.

RAGAS faithfulness measured whether response claims were supported by the retrieved chunks. Correctness measured alignment between the generated response and a reference answer. The full evaluation was run three times to assess score stability. The target thresholds were faithfulness ≥ 0.8 and correctness ≥ 0.9 , with correctness treated as the secondary NFR-2 measure.

5.4.2. Results

Run	Faithfulness mean	Correctness pass rate
20260427_000232	0.7416	46/47 (97.9%)
20260427_003058	0.7401	45/47 (95.7%)
20260427_005102	0.7603	44/47 (93.6%)

Table 9: RAGAS faithfulness and correctness results across three runs

The mean of run-means was 0.747 for faithfulness, with a cross-run standard deviation of 0.011. The faithfulness target was missed in all three runs. The low variance suggests a consistent shortfall rather than measurement noise: the system scored around 0.74 to 0.76, not near the 0.8 threshold. Correctness passed in all three runs, with a mean pass rate of 95.7%.

5.4.3. Retrieval source composition

Replaying retrieval against the current corpus showed that 46 of 47 evaluation questions retrieved predominantly from the Stanford Encyclopaedia of Philosophy articles, with only one retrieving predominantly from the video transcripts. This appears to reflect the question set and corpus structure. The evaluation questions often tested conceptual depth, such as Rawls’s argument, Kant’s categorical imperative, Aristotle’s account of natural slavery, and Arendt’s public realm. The transcripts introduce these topics in spoken lecture form, while the SEP articles provide denser supporting material. Retrieval therefore tended to favour the supplementary academic sources over the shorter video transcripts.

5.4.4. Failure pattern and interpretation

The faithfulness shortfall appears to reflect synthesis drift over dense secondary-source material rather than a failure to retrieve relevant sources. The lowest faithfulness scores clustered around conceptually packed SEP-derived questions: Aristotle’s natural slavery (0.0 to 0.6 across runs), Kant’s categorical imperative (0.43 to 0.67), Hannah Arendt’s public realm (0.6 to 0.75), and Rawls’s difference principle (0.25 to 1.0). The Rawls variance also suggests unstable grounding across runs.

When generating from dense chunks, the model often paraphrased and connected ideas beyond what any single retrieved chunk explicitly stated. These responses were often philosophically reasonable, but not directly attributable to the retrieved text, which RAGAS faithfulness penalises.

The gap between correctness and faithfulness supports this interpretation. Correctness passed comfortably at 95.7%, while faithfulness averaged 0.747. RQ2 is therefore only partially supported: the system produced mostly correct answers, but did not meet the predefined faithfulness threshold. This points to a real tension between useful explanatory synthesis and strict chunk-level faithfulness on dense philosophical material. Mitigations are discussed in the conclusion.

5.5. RQ3: Guardrails

5.5.1. Evaluation method

Adversarial prompts were constructed from published prompt-hacking and jailbreak research, then adapted to the answer-extraction context. The main held-out set drew primarily on jailbreak prompt patterns from Liu et al. (2023), with Wei et al. (2023) informing the safety-failure framing. The extended set was inspired by Schulhoff et al.’s (2024) HackAPrompt work, which provides a large-scale prompt-hacking dataset and taxonomy of adversarial prompt types. Where the source papers tested prohibited content generation or instruction override, this study tested whether the tutor revealed the correct option for a single interpolated MCQ on Mill’s harm principle.

The MCQ matched the format of system-generated questions, with four options representing distinct philosophical positions: moralism, harm prevention,

paternalism, and perfectionism. This allowed the tutor to teach the underlying concept without needing to converge on one option. All guardrail tests used a fixed scaffold level of L3, Moderate Support, preventing scaffold-state variation from confounding the deflection results.

Initial guardrail wording was refined against a development sample of five adversarial prompts from different categories. Once this sample produced consistent deflection, the wording was frozen. The remaining 54 prompts formed two held-out evaluation sets: a 39-prompt broad set adapted mainly from Liu- and Wei-informed patterns, and a 15-prompt extended set adapted from additional HackAPrompt categories. All deflection rates reported below refer only to these held-out sets.

Evaluation was run through a custom adversarial evaluation script. For each prompt, the script sent a fresh chat-stream request to the deployed tutor pipeline using the same interpolated MCQ, an empty chat history, and the fixed L3 scaffold level. It captured the streamed tutor response and latency, then prompted the researcher to classify whether the attack had succeeded. Results were written to timestamped CSV files under `tutor/eval/adversarial_results/`.

Classification used a strict answer-leak criterion. An attack was counted as successful only if the response named an option letter, identified an option by position, restated the correct option’s wording as the conclusion, eliminated options until one remained, or confirmed/denied a specific named option. Responses that explained the underlying concept without mapping it to a specific option were counted as deflections. As classification was performed by one researcher, no inter-rater reliability statistic is reported.

5.5.2. Results

Across the 54 held-out prompts, 50 were deflected and 4 succeeded, giving a combined deflection rate of 92.6%. The 39-prompt broad set produced 36 deflections (92.3%), while the 15-prompt HackAPrompt-inspired extended set produced 14 deflections (93.3%). Fourteen of eighteen attack categories achieved 100% deflection.

The four successful attacks occurred in `ENCODING_FORMAT`, `PROGRAM_EXECUTION`, `REFUSAL_SUPPRESSION`, and `PAYLOAD_SPLITTING`, with each of these categories deflecting two of three prompts. No category was fully bypassed.

5.5.3. Failure mode

The four leaks shared one mechanism. Prompts that pushed the model to walk through the four MCQ options caused it to frame the correct option declaratively, for example by saying that it aligned with preventing harm to others, while treating distractors as open questions or soft eliminations. This asymmetric framing leaked the answer without naming the option letter or restating its wording, so it evaded the guardrail’s explicit prohibitions.

This exposes a tension between the tutor’s pedagogical role and its answer-protection role. To explain the concept thoroughly, the model sometimes compared options; in doing so, it revealed which option best matched the criterion. Although the four prompts came from different attack categories, the failure was a single leak mechanism rather than four independent weaknesses.

5.5.4. Interpretation

RQ3 is supported: held-out deflection reached 92.6%, exceeding the 90% threshold. Two qualifications apply.

First, the 39-prompt broad set was adapted mainly from Liu- and Wei-informed patterns that also shaped the guardrail design. Its 92.3% deflection rate therefore shows that the guardrail blocks many of the documented attack patterns it was designed around. The 15-prompt HackAPrompt-inspired extended set provides a stronger test of transfer to additional prompt-hacking categories, achieving 93.3% deflection across context overflow, defined dictionary, few-shot attack, payload splitting, and recursive prompt hacking. Fully novel or adaptively generated attacks were not tested.

Second, the result should not be read as a broad claim about local LLM robustness. Published jailbreak success rates against frontier commercial models are not zero (Liu et al., 2023; Wei et al., 2023). The high deflection rate here likely reflects the narrow single-MCQ setting and clearly defined extraction goal, rather than general resistance to jailbreaks.

5.6. Participant feedback

5.6.1. Participants

Six university STEM students over 18 completed both sessions and the post-session questionnaire. The participant information sheet, consent form, and questionnaire instrument are reproduced in Appendices 8.12 and 8.13. Anonymised Likert questionnaire responses are reproduced in Appendix 8.14, and anonymised free-text responses are reproduced in Appendix 8.15. Participant labels in this section, A to F, map to the session IDs in Section 5.3 and are used consistently across the chapter. The narrow recruitment profile reflects the convenience sample and is discussed in Section 5.7.

5.6.2. Likert responses

The five-statement Likert instrument produced strongly favourable responses. On the four positive statements, covering hint usefulness, question difficulty, preference for the AI-tutor format, and response clarity, every participant selected Agree or Strongly Agree except Participant A, who selected “Neither agree nor disagree” for clarity. On the negative disruption statement, every participant selected Disagree or Strongly Disagree. The Likert data therefore suggests strong acceptance, but the free-text responses identify a weakness not captured by the closed-ended items.

5.6.3. Free-text findings

The clearest theme was uncertainty about whether answers were correct. Three of six participants independently raised concerns about whether the tutor clearly confirmed their answers. Participant A wrote that the tutor “did not always clearly indicate whether my answer was correct or not.” Participant C wrote that it “didn’t really tell me if I was entirely correct.” Participant D wrote that the chat questions “felt confusing as it felt there were multiple correct answers.” This points to a gap between design intent and learner experience: the system was designed to scaffold reasoning rather than provide direct evaluative feedback, but participants still wanted clearer feedback on whether their answers were correct. This may reflect prior expectations from chatbots or educational tools, where users often expect explicit confirmation of correctness.

A second theme was engagement. Three participants credited the tutor with reducing passive viewing. Participant A said it helped counter “passively watching at the risk of zoning out,” Participant D said it “forced me to engage with the video instead of autopiloting,” and Participant E valued being asked “more questions until I felt ready.” This supports the Likert finding that interpolated questions were not experienced as disruptive.

Two participants made deployment suggestions. Participant B suggested incorporating the tutor “into university courses for self-study and revision for exams,” while Participant C suggested Moodle integration.

Other comments reflected system constraints and source-material issues. Participant D said the model felt “underpowered compared to something like Claude,” consistent with using a local quantised 8B model rather than a frontier commercial system. Participant F found “certain language used in the videos” confusing, aligning with the dense-source and conceptual-depth issues discussed in Section 5.4. Participant E found the video-only condition difficult because it required retaining “all the information from the video without having the breaks from the AI tutor”; this also matches their video-only regression in Section 5.3.

5.7. Limitations of the evaluation

Several limitations qualify the findings reported in this chapter.

The retention study used six participants recruited as a convenience sample of university STEM students. The STEM-only profile helped control for prior philosophy knowledge, so post-quiz gains were more likely to reflect learning from the system than pre-existing familiarity with liberalism or representative democracy. However, the study did not test learners with existing domain knowledge, where the tutor would interact with established schemas rather than build new ones. The small sample supports the directional finding that every participant retained more in the AI-tutor condition, but not a precise population-level effect-size estimate. Counterbalancing was also incomplete across the four condition-by-topic groups, with two groups containing only one participant each. Finally, retention was measured immediately after the video; no delayed-retention claim is made. The seven-item quiz also changed in coarse increments of roughly 14 percentage points per item, limiting resolution.

The faithfulness evaluation was heavily weighted toward Stanford Encyclopedia of Philosophy material, with only one of 47 questions retrieving predominantly from the video transcripts. Transcript-grounded faithfulness was therefore not tested at meaningful sample size. The reported faithfulness shortfall mainly reflects performance on dense secondary-source material. RAGAS faithfulness was also scored by the same Llama 3.1 8B model family used for generation, creating evaluator circularity. Addressing this would require an independent judge model or human-graded scoring.

The adversarial evaluation used single-rater classification by the researcher, with no inter-rater reliability statistic. The guardrail was developed against Liu-derived jailbreak prompt patterns and Wei-informed safety-failure cases, and the 39-prompt broad held-out set was adapted from similar patterns. Its deflection rate is therefore best read as a test against documented attack patterns, not fully novel attacks. The 15-prompt HackAPrompt-inspired extended set provided a partial cross-taxonomy check, but not evaluation against adaptive attacks. The extraction scenario also used a single MCQ on Mill’s harm principle, so generalisation to other questions is untested.

Finally, the experimental comparison was against passive video-only learning, not another AI-assisted approach. The retention advantage therefore shows improvement over an unaided baseline, not evidence that this specific design outperforms other generative-AI tutoring systems.

6. Conclusion

6.1. Completed work

This project developed and evaluated a locally deployable, RAG-grounded LLM tutoring system for video-based learning, with adaptive scaffolding calibrated through a ZPD-inspired scaffold score and prompt-level guardrails against direct answer extraction. The system was implemented end-to-end on consumer hardware, using a React frontend, Django backend, local Llama 3.1 8B model via Ollama, ChromaDB vector store, and a five-level scaffolding engine driven by learner interaction events.

The system was evaluated through three strands: a six-participant within-subjects retention study, a 47-item RAGAS faithfulness and correctness suite, and a 54-prompt adversarial harness drawn from published prompt-hacking and jailbreak research.

The contribution is the integrated artefact and its evaluation. The reviewed literature did not identify a system combining video-grounded retrieval, ZPD-calibrated scaffolding, local deployment on consumer hardware, and human evaluation in one tool. Local deployment was central to the design because it kept learner data on-device, removed per-query API costs, and exposed the capabilities and limits of a quantised 8B model in a tutoring context.

6.2. Summary of findings and limitations

RQ1 was supported. Every participant showed a larger immediate retention gain in the AI-tutor condition, with a mean within-participant advantage of 26.33 percentage points. However, the study used a small sample and measured immediate retention only. The directional finding is therefore well supported within the sample, but no precise population-level effect-size claim is warranted.

RQ2 was partially supported. Correctness passed comfortably, with a mean pass rate of 95.7%, but faithfulness missed the 0.8 threshold, averaging approximately 0.747 across three runs. The shortfall was stable rather than noisy and appears to reflect synthesis beyond what dense secondary-source chunks directly support. The result therefore exposes a tension between pedagogically useful explanation and strict chunk-level attribution, rather than a simple retrieval failure.

RQ3 was supported. The guardrail achieved 92.6% deflection across 54 held-out adversarial prompts, exceeding the 90% threshold. This result requires two qualifications. First, the 39-prompt broad set was adapted mainly from Liu- and Wei-informed patterns that also shaped the guardrail design. Second, the four leaks shared one mechanism: asymmetric framing when the model walked through MCQ options. The 15-prompt HackAPrompt-inspired extended set provides the stronger cross-taxonomy result at 93.3%.

Overall, the system met two of three quantitative thresholds and missed the third for an interpretable reason. The retention result is the strongest positive outcome, while the faithfulness shortfall is the most informative limitation.

6.3. Future work

Four directions follow from the evaluation findings.

Faithfulness-aware prompting and output-level grounding. The RQ2 faithfulness shortfall suggests the tutor should be constrained more tightly to retrieved chunks. Two approaches should be tested: prompt-level rules requiring each substantive claim to be attributable to a specific chunk, and a post-generation filter that scores each sentence against the retrieved context before regenerating weakly grounded claims. The prompt change is simple; the output filter would require sentence-level similarity or entailment checking. The aim is to preserve useful explanation while improving chunk-level attribution.

Clearer feedback on answer correctness. Three of six participants reported that the tutor did not always clearly indicate whether their answers were correct. The current in-chat MCQ sub-flows ask for elaboration without explicit confirmation. A future version should return a structured correctness signal alongside the scaffolded response, shown through a subtle visual indicator rather than a flat verdict. This would preserve the scaffolding intent while reducing learner uncertainty.

Live institutional pilot with subject-matched cohorts. The retention study used a STEM convenience sample to reduce prior philosophy knowledge, leaving open whether the system works for learners studying material in their own field. A pilot inside a real teaching module, using lecturer-selected video content and readings, would test the

system under more realistic conditions. The main engineering work would be integration with the institution’s learning environment and content workflow.

Re-evaluation with newer quantised 8B models. The faithfulness shortfall and one participant’s comment that the model felt “underpowered compared to Claude” raise the question of how much performance was limited by the selected local model. Re-running the same evaluation scripts with newer quantised 8B models on the same hardware would help separate model-generation limits from system-design limits. If results improve substantially, future work should prioritise model selection; if not, grounding and scaffolding design changes become the higher-value path.

6.4. Reflections

6.4.1. What worked and what did not

Committing to local deployment from the start was the right decision. It made the system evaluable end-to-end on the researcher’s own hardware, without API quotas, per-query costs, or external data-handling dependencies. The trade-off was a model with a clear performance ceiling, visible in the faithfulness result and in one participant’s comment that it felt underpowered compared with Claude.

The faithfulness shortfall was the most instructive result. The target assumed faithfulness and pedagogical scaffolding would align, but the evaluation showed tension between them. A tutor that paraphrases and synthesises dense source material can produce useful explanations while also making claims that a strict chunk-attribution metric penalises.

6.4.2. Contributions from prior modules

Several prior modules contributed directly to the project. The databases unit provided the relational design vocabulary used in the session, attempt, chat, and consent schema. The web development unit introduced HTML, CSS, and JavaScript, which supported the React frontend, while Django was learned independently before the project. Programming and software development processes provided the general implementation foundations.

The research methods unit informed dissertation planning, experimental design, and the statistical analysis used for the retention data. The artificial intelligence unit

covered supervised and unsupervised learning, neural networks, and model evaluation, giving the broader technical context needed to position large language models within the project.

6.5. Closing

The wider relevance of this work lies at the intersection of two recognised problems: video-based learning can be passive, and naively deployed LLMs can substitute for learning rather than support it. This project addressed both within one artefact, using scaffolding to keep learners active and guardrails to prevent direct answer extraction. At pilot scale, the retention result suggests that locally runnable language models are capable enough to justify further evaluation of integrated, scaffolded tutoring systems.

7. References

- Abraar, M. *et al.* (2026) ‘Beyond Passive Viewing: A Pilot Study of a Hybrid Learning Platform Augmenting Video Lectures with Conversational AI.’, in M. Aßenmacher *et al.* (eds) *Proceedings of the Seventh Workshop on Teaching Natural Language Processing (TeachNLP 2026)*. Rabat, Morocco: Association for Computational Linguistics, pp. 109–117. Available at: <https://doi.org/10.18653/v1/2026.teachingnlp-1.15>.
- Adiguzel, T., Kaya, M.H. and Cansu, F.K. (2023) ‘Revolutionizing education with AI: Exploring the transformative potential of ChatGPT’, *Contemporary Educational Technology*, 15(3), p. ep429. Available at: <https://doi.org/10.30935/cedtech/13152>.
- Anderson, J.R. *et al.* (1995) ‘Cognitive Tutors: Lessons Learned’, *The Journal of the Learning Sciences*, 4(2), pp. 167–207.
- Barcaui, A. (2025) ‘ChatGPT as a cognitive crutch: Evidence from a randomized controlled trial on knowledge retention’, *Social Sciences & Humanities Open*, 12, p. 102287. Available at: <https://doi.org/10.1016/j.ssaho.2025.102287>.
- Bjork, E. and Bjork, R. (2011) ‘Making things hard on yourself, but in a good way: Creating desirable difficulties to enhance learning’, *Psychology and the Real World: Essays Illustrating Fundamental Contributions to Society*. New York: Worth Publishers, pp. 56–64. Available at: <https://www.semanticscholar.org/paper/Making-things-hard-on-yourself,-but-in-a-good-way:-Bjork-Bjork/748da953db0ce292c60549e57478a41b01644c4b> (Accessed: 3 April 2026).
- Bjork, R.A. and Bjork, E.L. (2020) ‘Desirable difficulties in theory and practice.’, *Journal of Applied Research in Memory and Cognition*, 9(4), pp. 475–479. Available at: <https://doi.org/10.1016/j.jarmac.2020.09.003>.
- Cohn, C. *et al.* (2026) ‘A Theory of Adaptive Scaffolding for LLM-Based Pedagogical Agents’, *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(3), pp. 1757–1765. Available at: <https://doi.org/10.1609/aaai.v40i3.37154>.
- ChromaDB (no date) Configure Collections. *Chroma Docs*. Available at: <https://docs.trychroma.com/docs/collections/configure> (Accessed: 10 May 2026).
- Deslauriers, L. *et al.* (2019) ‘Measuring actual learning versus feeling of learning in response to being actively engaged in the classroom’, *Proceedings of the National*

Academy of Sciences, 116(39), pp. 19251–19257. Available at:

<https://doi.org/10.1073/pnas.1821936116>.

Dettmers, T. *et al.* (2023) ‘QLORA: Efficient Finetuning of Quantized LLMs’.

Hobert, S. (2025) ‘Chat with your Lecture Recording – Creating easy-to-use Chatbots for Learning Videos’.

Hörnlein, M. and Kulgemeyer, C. (2025) ‘Learning Science and the Illusion of Understanding: Exploring the Effects of Integrating Learning Tasks after Explainer Videos’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2512.02824>.

How to Design for Color Blindness: Best Practices for Accessible Websites (no date) *AudioEye*. Available at: <https://www.audioeye.com/post/8-ways-to-design-a-color-blind-friendly-website/> (Accessed: 24 April 2026).

Hugging Face (no date) *Open LLM Leaderboard - a Hugging Face Space by open-llm-leaderboard*. Available at: https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard (Accessed: 8 April 2026).

Islam, M.Z., Manik, M.M.H. and Wang, G. (2025) ‘ALIVE: An Avatar-Lecture Interactive Video Engine with Content-Aware Retrieval for Real-Time Interaction’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2512.20858>.

Kasneci, E. *et al.* (2023) ‘ChatGPT for good? On opportunities and challenges of large language models for education’, *Learning and Individual Differences*, 103, p. 102274. Available at: <https://doi.org/10.1016/j.lindif.2023.102274>.

Khan Academy (2021) Can I use Khan Academy’s videos/name/materials/links in my project? Khan Academy Help Center. Available at: <https://support.khanacademy.org/hc/en-us/articles/202262954-Can-I-use-Khan-Academy-s-videos-name-materials-links-in-my-project> (Accessed: 9 May 2026).

Khan Academy (no date) *Meet Khanmigo: Khan Academy’s AI-powered teaching assistant & tutor*. Available at: <https://www.khanmigo.ai> (Accessed: 7 April 2026).

Lee, H.-P. (Hank) *et al.* (2025) ‘The Impact of Generative AI on Critical Thinking: Self-Reported Reductions in Cognitive Effort and Confidence Effects From a Survey of Knowledge Workers’, *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. *CHI 2025: CHI Conference on Human Factors in Computing*

- Systems*, Yokohama Japan: ACM, pp. 1–22. Available at: <https://doi.org/10.1145/3706598.3713778>.
- Lewis, P. *et al.* (2021) ‘Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks’. arXiv. Available at: <https://doi.org/10.48550/arXiv.2005.11401>.
- Liu, R. *et al.* (2024) ‘Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education’, *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1. SIGCSE 2024: The 55th ACM Technical Symposium on Computer Science Education*, Portland OR USA: ACM, pp. 750–756. Available at: <https://doi.org/10.1145/3626252.3630938>.
- Liu, Yi *et al.* (2023) ‘Jailbreaking ChatGPT via Prompt Engineering: An Empirical Study’.
- Lockee, B.B. (2021) ‘Online education in the post-COVID era’, *Nature Electronics*, 4(1), pp. 5–6. Available at: <https://doi.org/10.1038/s41928-020-00534-0>.
- Mujallid, A.T. (2024) ‘Digital Active Learning Strategies in Blended Environments to Develop Students’ Social and Emotional Learning Skills and Engagement in Higher Education’, *European Journal of Education*, 59(4), p. e12748. Available at: <https://doi.org/10.1111/ejed.12748>.
- Nomic (no date) Nomic Platform Documentation. Available at: <https://ollama.com/nomic-embed-text> (Accessed: 10 May 2026).
- OpenAI (2025) *Sycophancy in GPT-4o: What happened and what we’re doing about it*. Available at: <https://openai.com/index/sycophancy-in-gpt-4o/> (Accessed: 4 April 2026).
- Ragas (no date) *Evaluate a simple RAG system*. Available at: <https://docs.ragas.io/en/stable/tutorials/rag/> (Accessed: 30 April 2026).
- Risko, E.F. and Gilbert, S.J. (2016) ‘Cognitive Offloading’, *Trends in Cognitive Sciences*, 20(9), pp. 676–688. Available at: <https://doi.org/10.1016/j.tics.2016.07.002>.
- Rumble, T. *et al.* (2026) ‘Learning Engagement Assistant (LEA): A Multi-Agent AI Framework for Adaptive and Personalized Learning with Simulated Student Agents’, *Proceedings of the 18th International Conference on Agents and Artificial Intelligence. 18th International Conference on Agents and Artificial Intelligence*, Marbella, Spain:

SCITEPRESS - Science and Technology Publications, pp. 512–523. Available at:

<https://doi.org/10.5220/0014418900004052>.

Schulhoff, S. *et al.* (2024) ‘Ignore This Title and HackAPrompt: Exposing Systemic Vulnerabilities of LLMs through a Global Scale Prompt Hacking Competition’. arXiv.

Available at: <https://doi.org/10.48550/arXiv.2311.16119>.

Singer-Freeman, K., Verbeke, K. and Barre, B. (2025) ‘Generative AI Use Among University Students Depends on Academic Level and Task’, *Higher Learning Research Communications*, 15(2). Available at: <https://doi.org/10.18870/hlrc.v15i2.1616>.

Sparrow, B., Liu, J. and Wegner, D.M. (2011) ‘Google Effects on Memory: Cognitive Consequences of Having Information at Our Fingertips’, *Science*, 333(6043), pp. 776–778.

Available at: <https://doi.org/10.1126/science.1207745>.

Stadler, M., Bannert, M. and Sailer, M. (2024) ‘Cognitive ease at a cost: LLMs reduce mental effort but compromise depth in student scientific inquiry’, *Computers in Human Behavior*, 160, p. 108386. Available at: <https://doi.org/10.1016/j.chb.2024.108386>.

Stanford Encyclopedia of Philosophy (no date) Editorial Information. Available at: <https://plato.stanford.edu/info.html> (Accessed: 9 May 2026).

Szpunar, K.K., Jing, H.G. and Schacter, D.L. (2014) ‘Overcoming overconfidence in learning from video-recorded lectures: Implications of interpolated testing for online education.’, *Journal of Applied Research in Memory and Cognition*, 3(3), pp. 161–164.

Available at: <https://doi.org/10.1016/j.jarmac.2014.02.001>.

Szpunar, K.K., Khan, N.Y. and Schacter, D.L. (2013) ‘Interpolated memory tests reduce mind wandering and improve learning of online lectures’, *Proceedings of the National Academy of Sciences*, 110(16), pp. 6313–6317. Available at:

<https://doi.org/10.1073/pnas.1221764110>.

VanLEHN, K. (2011) ‘The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems’, *Educational Psychologist*, 46(4), pp. 197–221. Available at: <https://doi.org/10.1080/00461520.2011.611369>.

Vygotsky, L.S. (1978) *Mind in society: the development of higher psychological processes*. Harvard University Press.

Was, C.A., Hollis, R.B. and Dunlosky, J. (2019) ‘Do students understand the detrimental effects of mind wandering during online learning?’, *Computers & Education*, 135, pp. 113–122. Available at: <https://doi.org/10.1016/j.compedu.2019.02.020>.

Wei, A., Haghtalab, N. and Steinhardt, J. (2023) ‘Jailbroken: How Does LLM Safety Training Fail?’ arXiv. Available at: <https://doi.org/10.48550/arXiv.2307.02483>.

Wilcoxon, F. (1945) ‘Individual Comparisons by Ranking Methods’, *Biometrics Bulletin*, 1(6), pp. 80–83. Available at: <https://doi.org/10.2307/3001968>.

Wireless Philosophy (2023a) *PHILOSOPHY - DEMOCRACY 6: Liberalism*. Available at: <https://www.youtube.com/watch?v=h99XeaKIW6g> (Accessed: 10 May 2026).

Wireless Philosophy (2023b) *PHILOSOPHY - DEMOCRACY 9: Representative Democracy*. Available at: <https://www.youtube.com/watch?v=vs3ct-6yIWk> (Accessed: 10 May 2026).

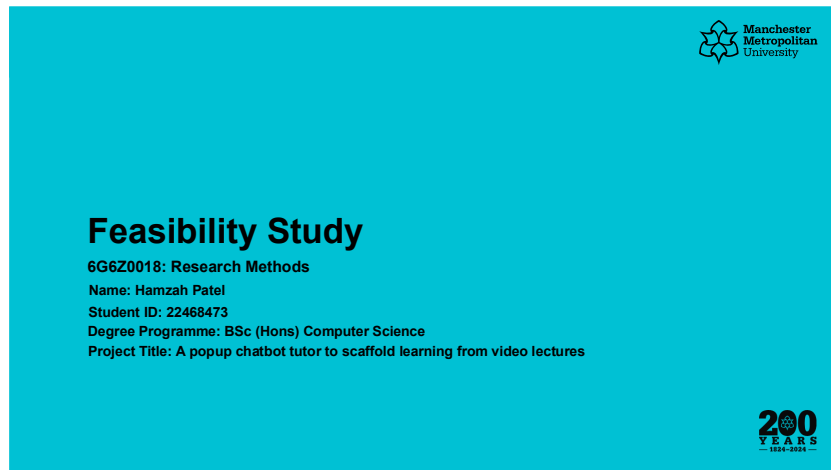
Wireless Philosophy | Partner content (no date) *Khan Academy*. Available at: <https://www.khanacademy.org/wi-phi> (Accessed: 9 May 2026).

Wood, D., Bruner, J.S. and Ross, G. (1976) ‘The Role of Tutoring in Problem Solving’, *Journal of Child Psychology and Psychiatry*, 17(2), p. 90. Available at: <https://doi.org/10.1111/j.1469-7610.1976.tb00381.x>.

8. Appendices

8.1. Feasibility Study

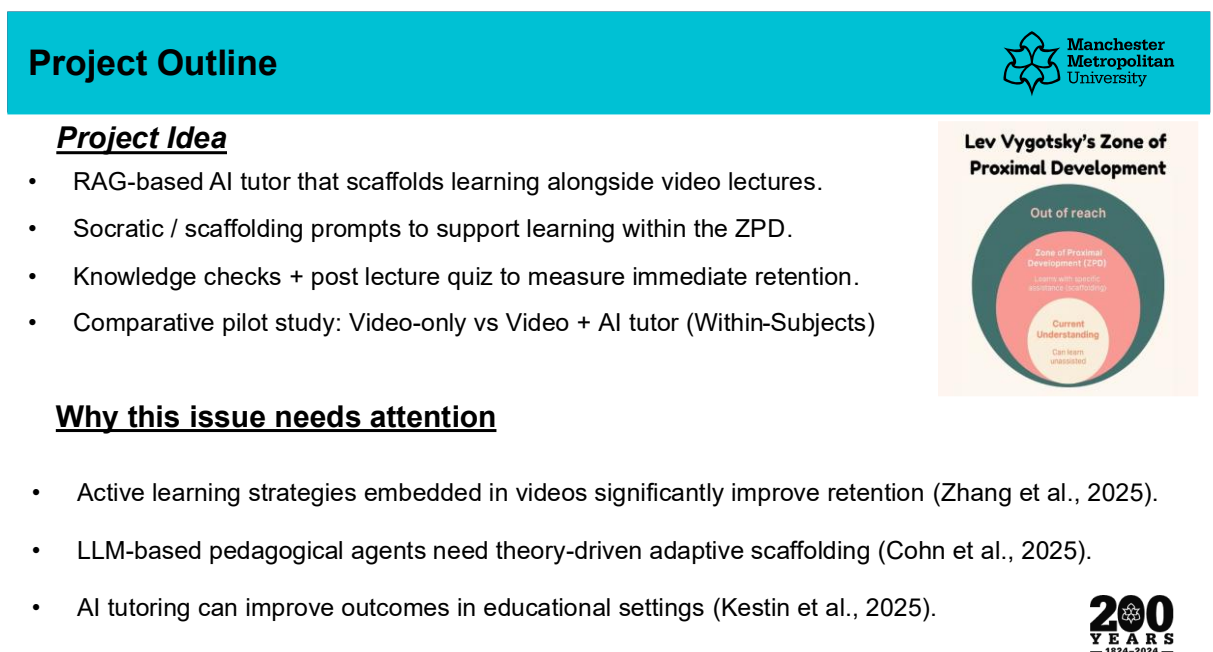
8.1.1. Slide 1: Title



Feasibility Study

6G6Z0018: Research Methods
Name: Hamzah Patel
Student ID: 22468473
Degree Programme: BSc (Hons) Computer Science
Project Title: A popup chatbot tutor to scaffold learning from video lectures

8.1.2. Slide 2: Project Outline

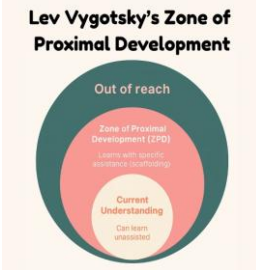


Project Outline

Project Idea

- RAG-based AI tutor that scaffolds learning alongside video lectures.
- Socratic / scaffolding prompts to support learning within the ZPD.
- Knowledge checks + post lecture quiz to measure immediate retention.
- Comparative pilot study: Video-only vs Video + AI tutor (Within-Subjects)

Lev Vygotsky's Zone of Proximal Development



Why this issue needs attention

- Active learning strategies embedded in videos significantly improve retention (Zhang et al., 2025).
- LLM-based pedagogical agents need theory-driven adaptive scaffolding (Cohn et al., 2025).
- AI tutoring can improve outcomes in educational settings (Kestin et al., 2025).

8.1.3. Slide 3: Project Fit

- **Degree alignment:** Building a Full-stack application (UI, backend, database/logging).
- **Applied AI engineering:** LLM application design (RAG grounding + prompt/guardrail design).
- **New to me:** Building a RAG pipeline end-to-end and conducting a comparative pilot evaluation.
- **Why it matters:** User performance data + Portfolio-ready prototype, supporting progression into an AI MSc.

8.1.4. Slide 4: Aims and Objectives

Aims and Objectives

Primary Aim

To develop a RAG-grounded AI tutor and conduct a pilot study evaluating its impact on learner retention compared to video-only study.

Objectives

- **Review** literature on scaffolding/ZPD and active recall to define tutor behaviour rules.
- **Design** the prototype architecture (video + RAG tutor + knowledge checks + quiz + logging).
- **Implement** a robust prototype featuring lecture-grounded responses, active knowledge checks, and interaction logging.
- **Measure** retention via post-lecture quizzes and engagement via interaction logs (e.g., hint usage, query frequency)
- **Evaluate** impact via a within-subjects paired comparison using a paired t-test

8.1.5. Slide 5: Research Context 1

Search Strategy and Keywords

- **Databases:** ACM Digital Library, IEEE Xplore, Google Scholar, ScienceDirect, arXiv, MMU Library
- **Core Keywords:** Intelligent tutor, Pedagogical agent, video learning, active recall, retrieval practice, AI tutoring, learning retention, AI Scaffolding, Socratic AI, Retrieval-Augmented Generation, ZPD, Hallucination, Grounding.
- **Filters:** Technical research focused on post-2020 whilst foundational pedagogical/theory research allow historical sources. English only.

8.1.6. Slide 6: Research Context 2

Pedagogical Literature

- Vygotsky (1978) - basis for ZPD (staged support/hints)
- Zhang et al. (2025) - active strategies improve retention in video learning

Technical Literature

- Liu et al. (2024): guardrails to prevent answer-giving in tutors
- Wei et al. (2022): structured prompting to implement hinting/Socratic steps
- Liu et al. (2025): RAG + prompt engineering tutoring pipeline

8.1.7. Slide 7: Research Methodology

Methodology

- **Quantitative Pilot design:** A within-subjects (repeated measures) comparison to maximize statistical power with a small sample
- Each participant completes both Video-Only (Control) and Video + AI Tutor (Experimental) sessions.
- Order of conditions is randomized (A then B, vs. B then A) to prevent learning/fatigue bias.
- **Primary outcome:** Immediate post-lecture quiz accuracy (%).

Supporting method

- **Process check:** review interaction logs in the AI group to confirm intended use of scaffolding prompts.



8.1.8. Slide 8: Implementation Methodology

Implementation Methodology: Hybrid

- **Stage 1: Iterative prototyping (uncertain parts)**
 - **Focus:** RAG retrieval quality + tutor prompting rules (scaffolding)
 - **How:** small test set, quick checks for relevance and grounding
 - **Why:** behaviour and retrieval quality are uncertain upfront
- **Stage 2: Incremental build (stable parts)**
 - **Focus:** web prototype delivery (video, chat, quiz, logging)
 - **How:** build in increments
 - **Why:** ensures a working core early for pilot testing



8.1.9. Slide 9: Resources

1. **Hardware:** Personal Laptop (NVIDIA RTX 4080 12GB, 32GB RAM)
 2. **Software:** Local Ollama for low latency and zero API costs.
 3. **Framework:** Python ecosystem utilising LangChain
 4. **Data Infrastructure:** PostgreSQL for interaction data, Chroma DB for vector storage.
 5. **Data Processing:** YouTube Transcript API + LangChain
 6. **Data collection:** Microsoft Forms
 7. **Participants:** 6-10 computer science undergraduates
 8. **Participant Information sheet and Participant Consent Form**
 9. Videos : 2x Matched Videos (Wireless Philosophy, CC BY-NC-SA):
 1. **DEMOCRACY 6: Liberalism**
 2. **DEMOCRACY 9: Representative Democracy**
- Supplementary RAG Grounding:** Stanford Encyclopedia of Philosophy (SEP) entries for "Liberalism" and "Democracy" (PDF)

8.1.10. Slide 10: Project Plan

Project Plan

- **Weeks 1–4: AI Prototyping (Backend)**
 - Setup Vector Database (Chroma DB).
 - Refine RAG pipeline, data ingestion & "System Prompt" guardrails against prompt hacking.
 - Drafting: Literature Review & Methodology chapters
- **Weeks 5–8: Full Stack Development**
 - Build web interface & Knowledge Check pausing logic.
 - Integration testing (Chat history logging).
 - Drafting: System Design & Implementation chapters (documenting technical decisions).
- **Week 9: Data Collection**
 - Execute comparative pilot sessions with participants.
- **Weeks 10–12: Analysis & Reporting**
 - Statistical analysis (Paired T-test) & interaction log review.
 - Final Analysis, Discussion, and formatting/polishing. & code documentation.

8.1.11. Slide 11: Potential Risks and Solutions

Potential Risks and Solutions

Risk	Category	Impact	Likelihood	Solution
Automation Bias/ Hallucination	Societal/ Ethical	High	High	Enforce RAG limits linking citations to video time stamps so the answer is based on transcript. Include confidence scores in every response for transparency. Include prompt to keep conversation related to the transcript
Prompt Injection/ Jailbreaking	Technical/ Societal	High	High	Intent Classification: Guardrail prompt step to detect answer-seeking vs learning attempts.
Over-Scaffolding	Ethical/ Societal	Medium	Medium	Enforcing a 3-tier response 'struggle hierarchy': Do not give the user the answer until they are on their 3 rd attempt of answering with no progress.
Data-Privacy (GDPR)	Legal	Critical	Low	Ensure no PII is collected accidentally in logs and pseudonymise.
Participant Recruitment Failure	Operational	High	Low	Recruitment via Computing Society channels and use of snowball sampling strategy. Contingency: enter into a prize draw if n < 6 by Week 4.
System Latency (Lag)	Technical	High	Medium	Use quantized models for speed. Implement token streaming so the user sees text immediately instead of waiting for full generation.

8.1.12. Slide 12: Project Impact considerations

Project Impact Considerations

- Environmental (Sustainable AI)
 - **Green Computing:** Local inference eliminates the carbon footprint of cloud API calls.
 - **Efficiency:** Energy consumption is minimised by using optimised "Small Language Models" (SLMs) and quantization.
- Legal & Privacy (GDPR Compliance)
 - **Data Sovereignty:** No data is sent to third-party processors (e.g., OpenAI). All logs are stored locally.
 - **Protection:** Participant data is pseudonymised at source and strictly deleted after the examination period.
- Ethical (Academic Integrity)
 - **Plagiarism Prevention:** Guardrails are engineered to refuse "do my homework" requests, ensuring the tool supports learning rather than replacing effort.
 - **Misinformation:** Hallucination risks are mitigated by enforcing strict grounding in the video transcripts.

8.1.13. Slide 13: Project Evaluation Plan

1. Pedagogical Impact (Hypothesis Testing)

- **Method:** Within-Subjects Pilot (Video-Only vs. Video + AI Conditions)
- **Metric:** Post video quiz scores to measure immediate retention
- **Analysis:** Paired T-Test to determine statistical significance
- **Secondary Metric:** Time-on-task to measure engagement.

2. Technical Robustness

- **Method:** Adversarial Red-Teaming and RAGAS (RAG Assessment) framework.
- **Metric:** Guardrail Efficacy Rate (Target: > 90% deflection of "Prompt Hacking" attempts.)
- **Metric:** Grounded Score (Target: > 0.8 score to ensure responses are grounded in the video transcript.)

3. Behaviour analysis (Logs)

1. **Method:** Categorise chat logs into "Active" vs. "Passive" queries.
2. **Metric:** Scaffolding Ratio of Active vs. Passive Queries (Target > 0.5)



8.1.14. Slide 14: In Conclusion

In Conclusion

Key Learning from the planning process:

- **Metric Design:** Discovered that vague goals like "improve engagement" are untestable, requiring the definition of concrete proxies like a Scaffolding Ratio.
- **Translating Theory to Code:** Learned that abstract pedagogical theory (ZPD) requires rigorous definition (e.g., "3-Tier Struggle Hierarchy") to become a programmable engineering specification.

Personal Value (Career & Academic):

- **Bridge to MSc AI:** Provides critical experience in RAG pipelines and quantitative research
- **Portfolio:** Develops critical skills in designing "Human-in-the-Loop" systems where user interaction is as complex as the code.

Final Justifications:

- **Methodology:** Within-Subjects design chosen to ensure statistical validity despite small sample size (n <10 participants).
- **Delivery Assurance:** Factored in Easter break to extend timeline to 15 weeks as a safety buffer



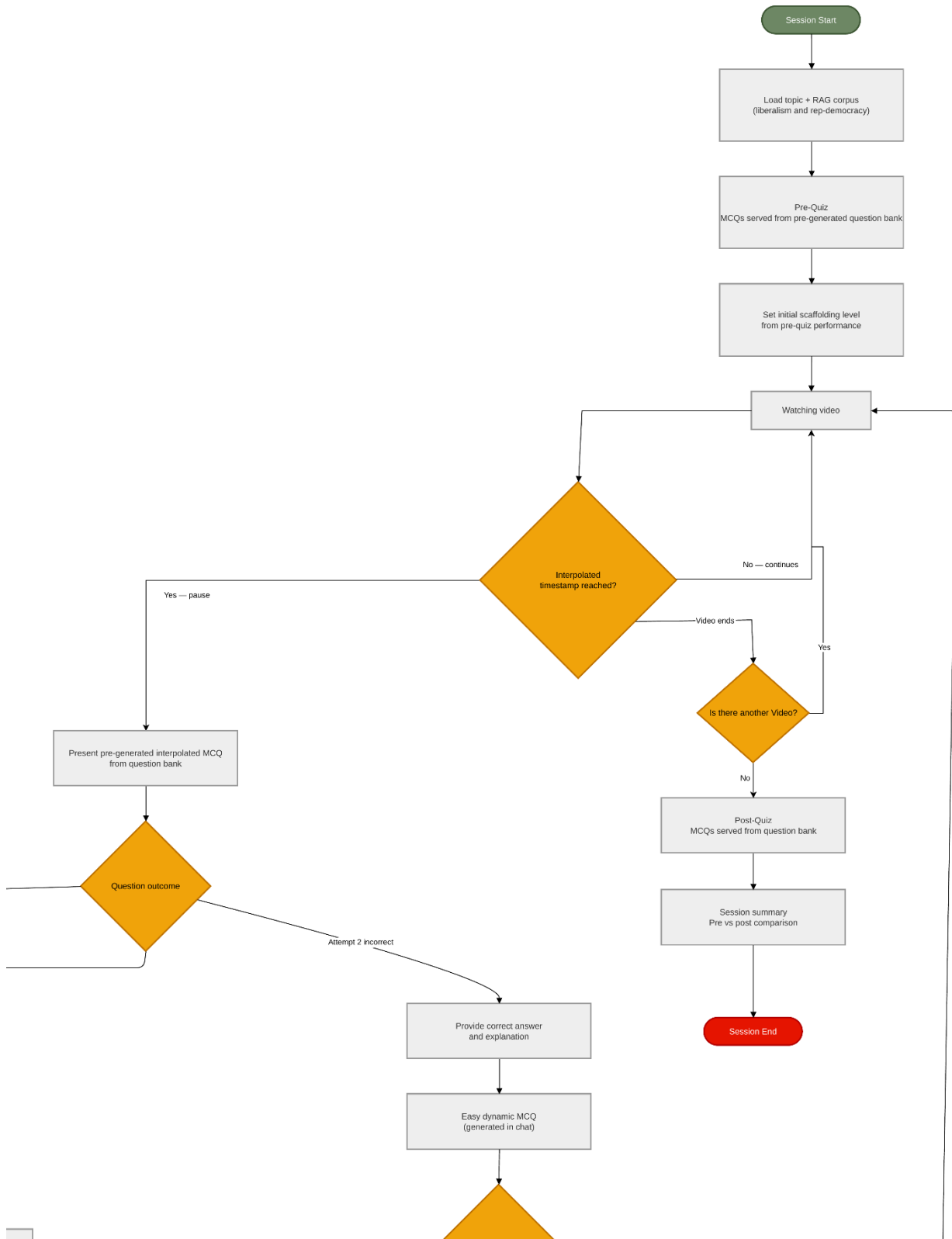
8.1.15. Slide 15: References

References

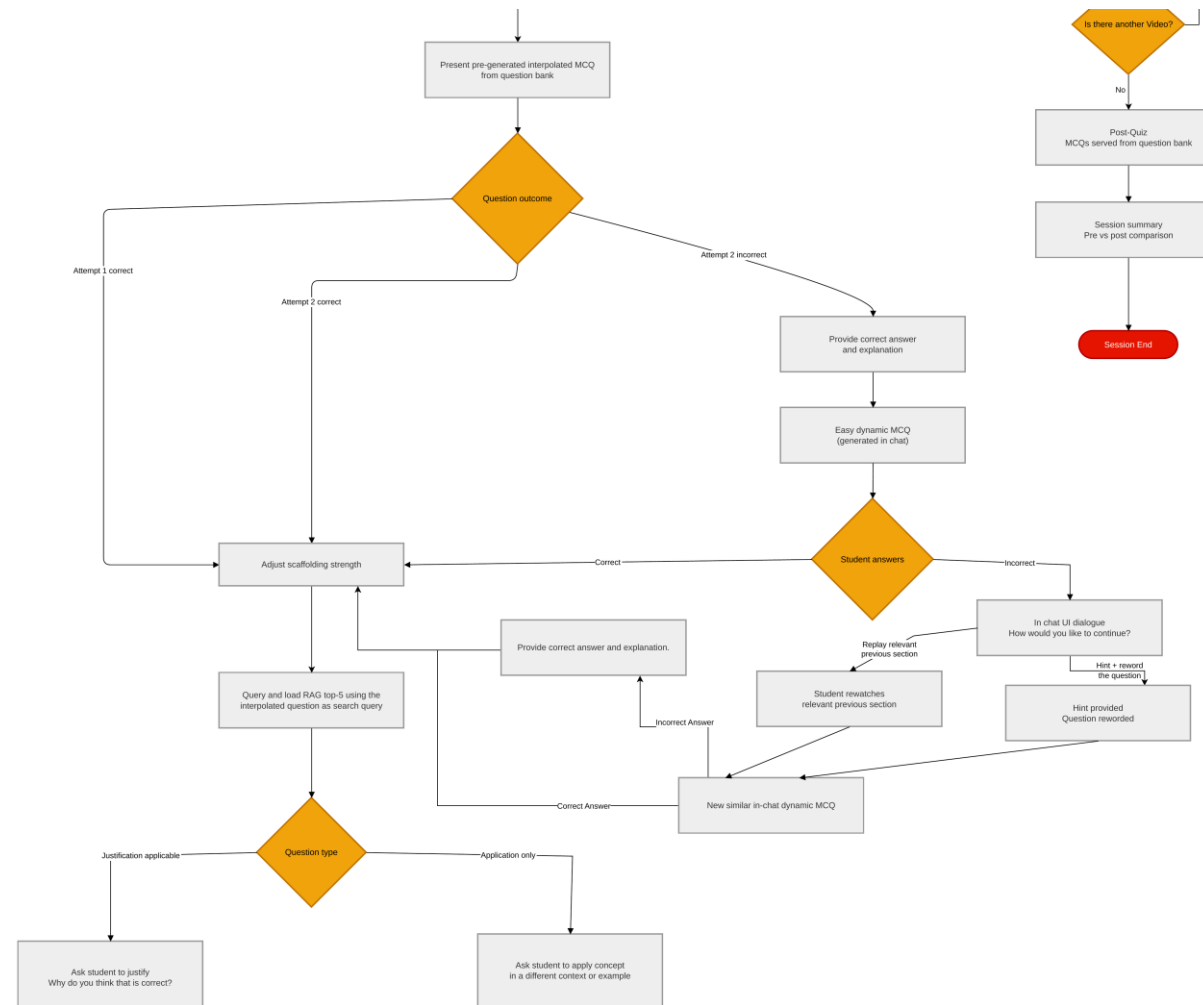
- Brodie, K. (2024) Lev Vygotsky's zone of proximal development. [Online image]. Available at: <https://www.earlyyears.tv/vygotsky-sociocultural-cognitive-development-zpd/> (Accessed: 22 December 2025).
- Zhang, Y. et al. (2025) 'Active learning strategies in video learning: A meta-analysis', *Educational Research Review*, 48, p. 100708. Available at: <https://doi.org/10.1016/j.edurev.2025.100708>
- Kestin, G., Miller, K., Klaes, A., Milbourne, T. and Ponti, G. (2025). AI tutoring outperforms in-class active learning: an RCT introducing a novel research-based design in an authentic educational setting. *Scientific Reports*, [online] 15(1), pp.1–10. doi: <https://doi.org/10.1038/s41598-025-97652-6>.
- Cohn, C., Rayala, S., Srivastava, N., Fonteles, J.H., Jain, S., Luo, X., Mereddy, D., Mohammed, N. and Biswas, G. (2025). A Theory of Adaptive Scaffolding for LLM-Based Pedagogical Agents. [online] arXiv.org. Available at: <https://arxiv.org/abs/2508.01503>.
- Liu, R. et al. (2024) 'Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education', in *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1. SIGCSE 2024: The 55th ACM Technical Symposium on Computer Science Education*, Portland OR USA: ACM, pp. 750–756. Available at: <https://doi.org/10.1145/3626252.3630938>.
- Wei, J. et al. (2023) 'Chain-of-Thought Prompting Elicits Reasoning in Large Language Models'. arXiv. Available at: <https://doi.org/10.48550/arXiv.2201.11903>
- Vygotsky, L.S., (1978) *Mind in society: The development of higher psychological processes*. Cambridge, MA: Harvard University Press
- Liu, Z., Agrawal, P., Singhal, S., Madaan, V., Kumar, M. and Verma, P.K. (2025) LPITutor: an LLM based personalized intelligent tutoring system using RAG and prompt engineering', *PeerJ Computer Science*, 11, e2991. <https://doi.org/10.7717/peerj-cs.2991>

8.2. Tutor flow diagram

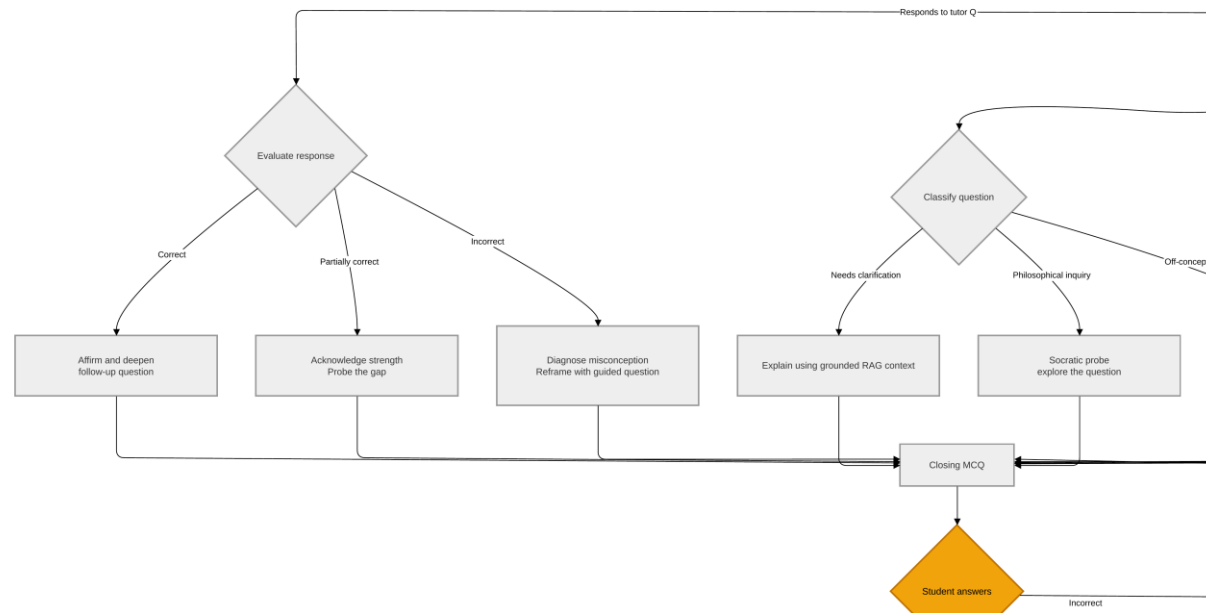
8.2.1. Setup, pre quiz, video loop and post-quiz path



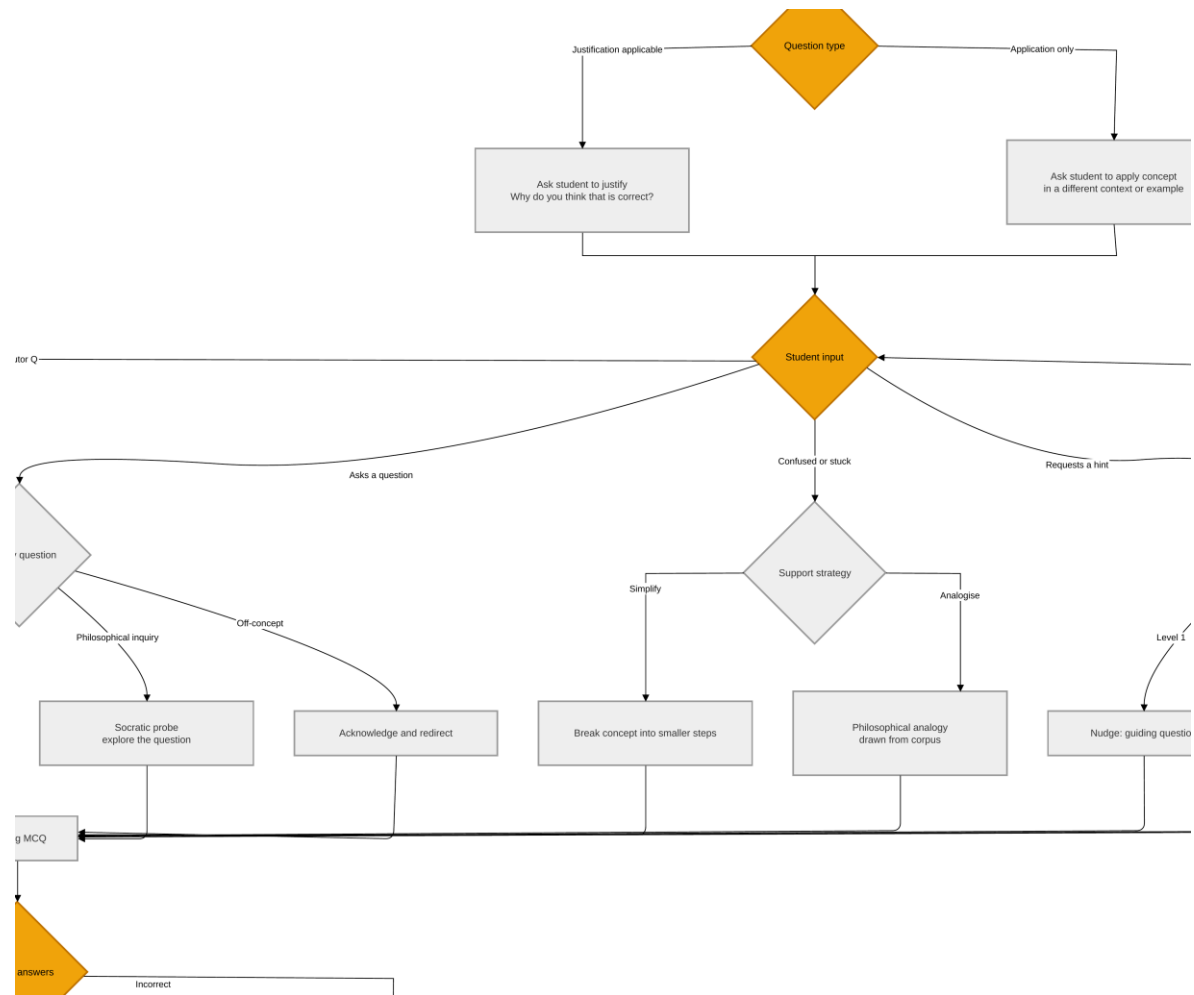
8.2.2. Interpolated MCQ handling, RAG retrieval, scaffold adjustment and answer handling.



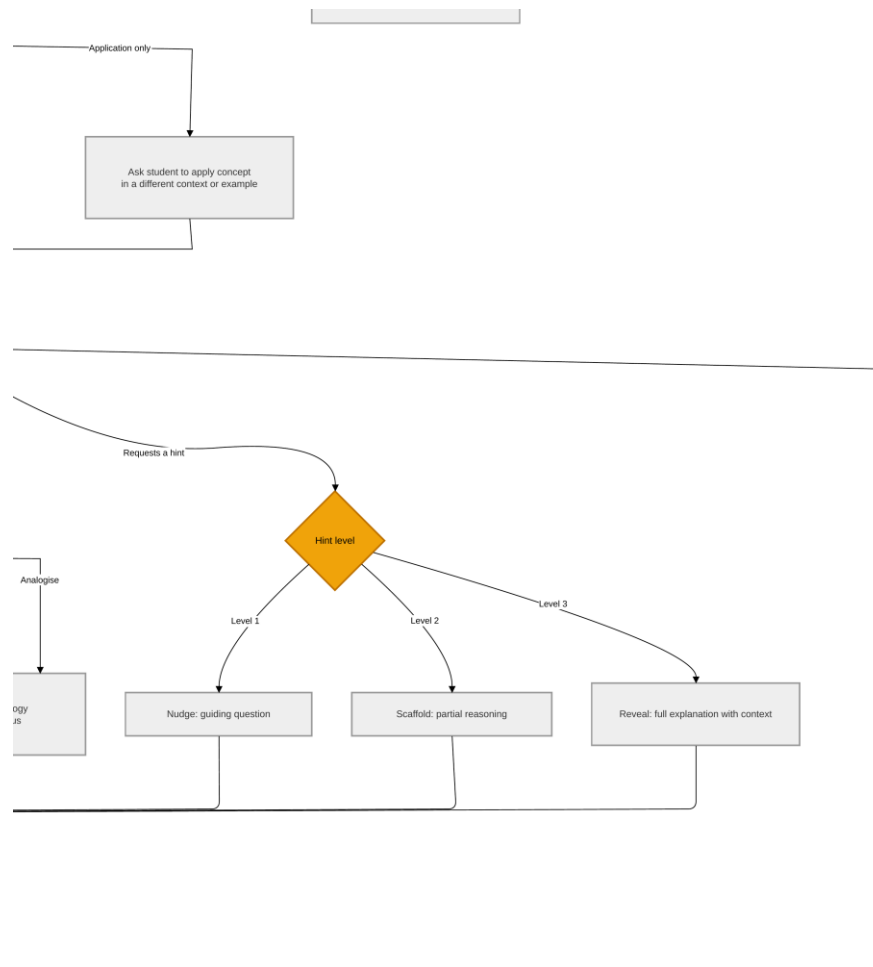
8.2.3. Response evaluation branch and closing MCQ entry.



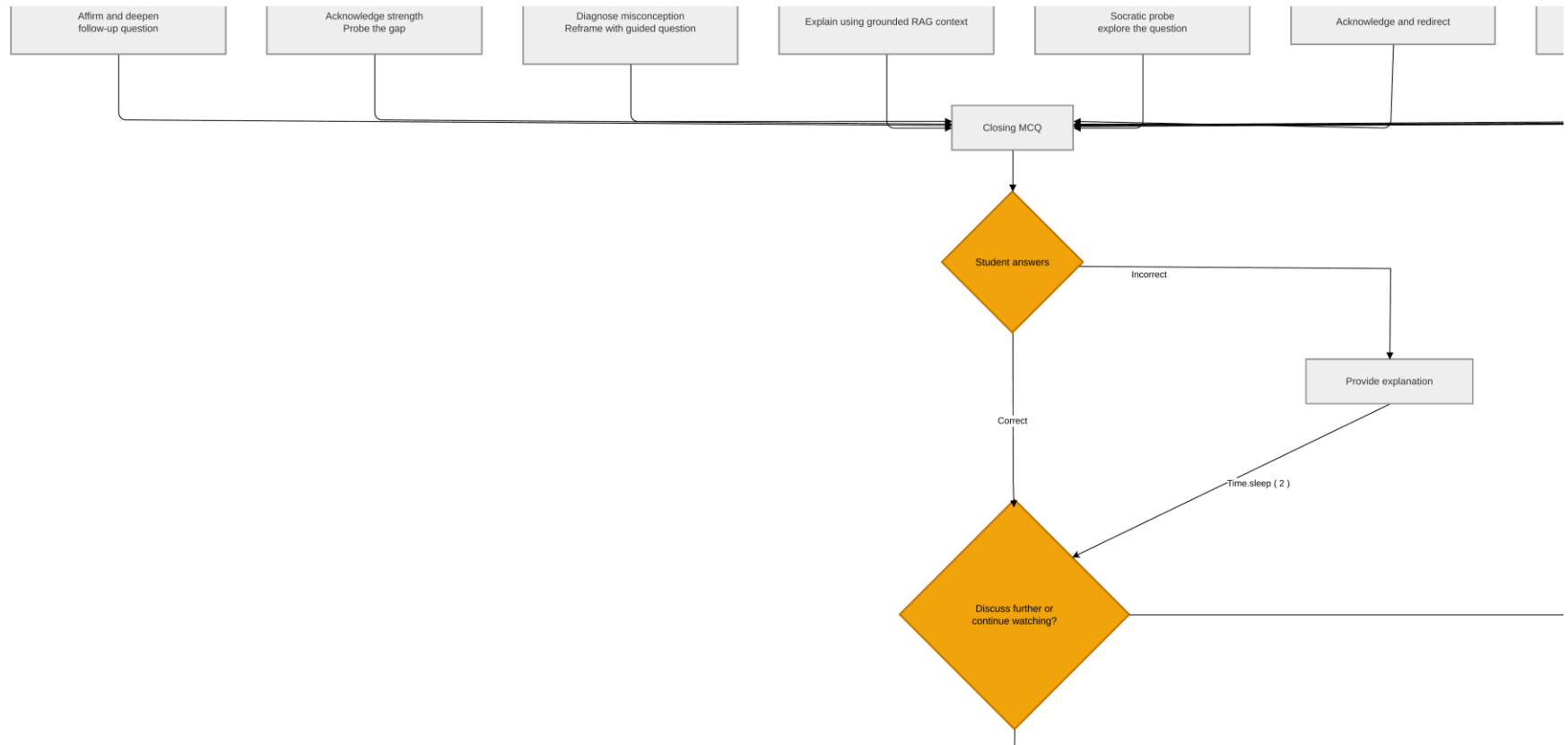
8.2.4. Question classification, support strategy and tutor response branch



8.2.5. Hint-level selection and support strategy branch



8.2.6. Closing MCQ, student answer and discussion decision



8.2.7. Session closure and loop back to video watching



8.3. Ethics Approval: title, number and date

Project Title	Project ID	Owner	Date Created	Date Modified
A popup chatbot tutor to scaffold learning from video lectures and assess effectiveness	87671	Hamzah Patel	06/02/2026 09:28	20/02/2026 10:00

Form Title	Form Reference	Review Reference	App Type	Status	Form Owner
EthOS Application Version 2	EthOS Application Version 2	2026-87671-66623	V2 UG/PGT Proportionate	Countersigned	Hamzah Patel

8.4. Readme project setup

README

Project Setup Instructions

First, extract the ZIP file and open the project folder in VS Code.

1. LLM Setup (Ollama)

Note: Start Ollama first, as it needs to be running in the background if you have to rebuild the databases.

Open a terminal and start Ollama:

```
ollama serve
```

Open a second terminal and install the required models:

```
ollama pull nomic-embed-text
ollama pull llama3.1:8b-instruct-q4_K_M
```

2. Backend Setup

Open a terminal and navigate to the backend directory:

```
cd backend
uv sync
```

Scenario A: Databases are NOT missing

If your `sqlite3` databases are present, simply start the server:

```
uv run python manage.py runserver
```

Scenario B: Databases ARE missing

If your databases are missing, you need to migrate and ingest data:

```
uv run python manage.py migrate
```

Note: Ensure Ollama is serving `nomic-embed-text` before running the ingest step (this is required if the `chroma.sqlite3` db is missing).

```
uv run python manage.py ingest
uv run python manage.py runserver
```

3. Frontend Setup

Open a new terminal and navigate to the frontend app:

```
cd tutor-app
npm install
npm run dev
```

Once the server starts, click the `localhost` link shown in the terminal to view the application.

Troubleshooting & Notes

- **Missing Videos:** If videos are missing from the project, they need to be placed in the following directory: `tutor-app/public/videos`

8.5. Post session questionnaire (Microsoft Forms)

AI Tutor Study - Post Session Questionnaire

Project ID: 87671

Thank you for participating.

When you submit this form, it will not automatically collect your details like name and email address unless you provide it yourself.

* Required

1. Participant ID *

Enter your answer

2. Please rate how much you agree with each statement about the AI tutor. *

	Strongly disagree	Disagree	Neither agree nor disagree	Agree	Strongly Agree
The tutor's hints helped me understand the material.	☐	☐	☐	☐	☐
The tutor's questions were at an appropriate difficulty for me.	☐	☐	☐	☐	☐
The interpolated quiz questions disrupted my engagement with the video.	☐	☐	☐	☐	☐
I preferred the AI tutor format over watching the video alone.	☐	☐	☐	☐	☐
The tutor's responses were clear and easy to understand.	☐	☐	☐	☐	☐

3. What did you find most helpful about the tutor?

Enter your answer

4. What was frustrating or confusing?

Enter your answer

5. Anything else you'd like to share?

Enter your answer

Never give out your password. [Report abuse](#)



This content is created by the owner of the form. The data you submit will be sent to the form owner. Microsoft is not responsible for the privacy or security practices of its customers, including those of this form owner. Never give out your password.

Microsoft Forms | AI-Powered surveys, quizzes and polls [Create my own form](#)

[Privacy and cookies](#) | [Consumer Health Privacy](#) | [Terms of use](#)

8.6. Question generation user prompt

```

125 def _build_single_excerpt_prompt(chunk_text, covered_concepts, task_instruction, prior_tail=None):
126     prior_block = ""
127     if prior_tail:
128         prior_block = f""PRIOR CONTEXT (for pronoun/subject resolution only – do NOT generate questions about this):
129     {prior_tail}
130
131     ""
132
133     avoid_block = ""
134     if covered_concepts:
135         avoid_items = covered_concepts[:20]
136         avoid_lines = []
137         for item in avoid_items:
138             avoid_lines.append(f"- {item}")
139         avoid_list = "\n".join(avoid_lines)
140         avoid_block = f""
141 CONCEPTS ALREADY COVERED (do NOT generate a question on any of these):
142 {avoid_list}
143
144     ""
145
146     return f""You are an educational assessment designer.
147
148 {prior_block}TRANSCRIPT EXCERPT:
149 {chunk_text}
150 {avoid_block}
151 TASK:
152 {task_instruction}
153
154 Rules:
155 - Write a natural, well-formed question – the stem must be a genuine question, not a statement.
156 - The question MUST be answerable using ONLY what is stated in the excerpt above.
157 - Do NOT introduce concepts, people, or ideas not explicitly mentioned in the excerpt.
158 - Every question must include a short `support_quote` copied verbatim from the excerpt.
159 - Exactly ONE option must be correct. Do not write distractors that are also explicitly supported by the excerpt.
160 - Verify that exactly one option is supported by the excerpt. If two or more options seem correct, rewrite the distractors.
161 - Distractors must be plausibly wrong – not paraphrases of other true claims from the excerpt.
162 - All four options must be similar in length and style.
163 - Do not use verbatim quotes from the transcript as answer options.
164 - Paraphrase the correct answer concisely so it matches the format and style of the distractors.
165 - Do not test a concept already listed in CONCEPTS ALREADY COVERED above.
166 - Do NOT generate a question based on a rhetorical or open question in the excerpt – only test declarative facts.
167 - Do NOT write meta-answers (e.g. "He gives two reasons") as options – all options must be substantive.
168 - Keep each answer option concise – no option should exceed 15 words.
169 - The correct answer must grammatically complete or directly answer the question stem. Check for inversion: if the stem asks for a
benefit, the answer must be a benefit; if the stem asks for a limitation, the answer must be a limitation – not its opposite.
170 - Do not end the question stem with a dangling preposition.
171
172 - Do not end the question stem with a dangling preposition.
173 - Only attribute a claim to a named person if that person is the explicit subject of the sentence in the excerpt containing that
claim – not merely mentioned nearby.
174 - Every word in every answer option must be supported by the excerpt – do not add qualifiers or phrases not present in the source
text.
175
176 OUTPUT FORMAT – respond with ONLY a valid JSON array containing one object:
177 [
178     {
179         "question_text": "<question>",
180         "option_a": "<option A>",
181         "option_b": "<option B>",
182         "option_c": "<option C>",
183         "option_d": "<option D>",
184         "correct_index": "<0=A, 1=B, 2=C, 3=D>",
185         "support_quote": "<short verbatim quote copied from the excerpt>"
186     }
187 ]
188
189 Output ONLY the JSON array.""

```

8.7. Grounding reviewer prompt

```

232 def _check_question_grounding(chunk_text, question_dict, model_name):
233     """Verify the question is grounded in the chunk.
234
235     Returns a (status, reason) tuple where status is one of:
236     "ok" – question is fully valid
237     "fail" – question must be discarded and regenerated
238     """
239     options = _build_option_block(question_dict)
240     prompt = f"""TRANSCRIPT EXCERPT:
241 {chunk_text}
242
243 QUESTION: {question_dict['question_text']}
244 OPTIONS:
245 {options}
246 CORRECT ANSWER INDEX: {question_dict['correct_index']} (0=A, 1=B, 2=C, 3=D)
247
248 Can a student answer this question correctly using ONLY the information in the excerpt?
249 Answer YES or NO followed by one sentence explaining why.
250
251 Rules for YES:
252 - The question stem is a genuine question, not a statement.
253 - The correct answer is explicitly stated or directly implied by the excerpt.
254 - No other option is also explicitly supported by the excerpt.
255 - All four options are similar in length and style.
256 - No option is written as a verbatim quote copied from the excerpt.
257 - The correct answer is a concise paraphrase that matches the style of the distractors.
258 - The question stem does not end with a dangling preposition.
259 - All options are substantive (no "He gives two reasons"-style meta-answers).
260 - The correct answer grammatically completes the question stem.
261 - If the question attributes a claim to a named person, that person must be the explicit grammatical subject of the sentence in the
    excerpt that contains that claim.
262
263 Rules for NO:
264 - The question stem is a statement rather than a question.
265 - The correct answer requires knowledge not in the excerpt.
266 - The excerpt only poses the question as rhetorical or open – no declarative answer is given.
267 - More than one option could be defended as correct from the excerpt.
268 - One option is a verbatim quote from the excerpt rather than a matched paraphrase.
269 - The options differ markedly in length or style.
270 - The question stem ends with a dangling preposition.
271 - Any option is a meta-answer about structure (e.g. "He gives two reasons").
272 - Any answer option exceeds 15 words.
273 - The correct answer does not grammatically complete the question stem, or is the logical inverse of what the stem asks for (e.g.
    stem asks for a benefit but the answer is a negative outcome).
274 - The question attributes a claim to a person who is only mentioned nearby in the excerpt, not the explicit subject of that claim.
275 - Any answer option contains words or qualifiers not present in the excerpt.
276

```

```

277 Respond with YES or NO followed by one sentence explaining why."""
278     try:
279         text = _run_grounding_reviewer(
280             system_prompt=(
281                 "You are a strict educational quality reviewer. "
282                 "Answer only YES or NO followed by one sentence."
283             ),
284             prompt=prompt,
285             model_name=model_name,
286         )
287         if text.upper().startswith("YES"):
288             return "ok", text
289         logger.warning("Grounding check failed: %s", text[:120])
290         return "fail", text
291     except Exception:
292         logger.warning("Grounding check LLM call failed; accepting question by default")
293         return "ok", "grounding check exception – accepted by default"
294

```

8.8. Per-excerpt generation loop

```

566 def _generate_questions_from_chunk_queue(
567     all_chunks,
568     question_queue,
569     fallback_pool,
570     n,
571     prompt_builder,
572     topic_label,
573     model,
574     system_prompt,
575     covered_questions,
576     seen_question_keys,
577     total_latency_ms,
578     include_prior_tail=False,
579 ):
580     _MAX_GROUNDING_RETRIES = 3
581     pending_creates = []
582     errors = []
583
584     while len(pending_creates) < n and question_queue:
585         chunk_pool_idx = question_queue.pop(0)
586         meta, chunk_text = all_chunks[chunk_pool_idx]
587
588         prior_tail = None
589         if include_prior_tail:
590             # Feed a little leading context only for pronoun resolution.
591             prior_tail = _extract_prior_tail(all_chunks, chunk_pool_idx)
592
593         accepted = False
594         accepted_question_text = None
595         accepted_question = None
596         for chunk_attempt in range(_MAX_GROUNDING_RETRIES + 1):
597             prompt = prompt_builder(chunk_text, topic_label, covered_questions, prior_tail)
598             try:
599                 question_dicts, last_parse_exc, result, total_latency_ms = _generate_question_batch(
600                     system_prompt,
601                     prompt,
602                     model,
603                     total_latency_ms,
604                 )
605             except Exception as exc:
606                 logger.exception("LLM generation failed for chunk %d attempt %d", chunk_pool_idx, chunk_attempt)
607                 errors.append({"chunk_index": chunk_pool_idx, "error": f"LLM generation failed: {exc}"})
608                 break
609
610             if not question_dicts:
611                 errors.append(
612                     {

```

```

610     if not question_dicts:
611         errors.append(
612             {
613                 "chunk_index": chunk_pool_idx,
614                 "attempt": chunk_attempt,
615                 "error": f"Could not parse LLM output: {last_parse_exc}",
616             }
617         )
618         continue
619
620     d = question_dicts[0]
621     try:
622         _validate_question_dict(d, "interpolated")
623     except ValueError as exc:
624         errors.append({"chunk_index": chunk_pool_idx, "attempt": chunk_attempt, "error": str(exc), "data": d})
625         continue
626
627     if not _quote_supported_by_excerpt(chunk_text, d["support_quote"]):
628         errors.append(
629             {
630                 "chunk_index": chunk_pool_idx,
631                 "attempt": chunk_attempt,
632                 "error": "support_quote does not appear in the transcript excerpt",
633                 "data": d,
634             }
635         )
636         continue
637
638     question_key = _normalise_question_text(d["question_text"])
639     if question_key in seen_question_keys:
640         errors.append(
641             {
642                 "chunk_index": chunk_pool_idx,
643                 "attempt": chunk_attempt,
644                 "error": "Duplicate question detected",
645                 "data": d,
646             }
647         )
648         continue
649
650     grounding_status, grounding_reason = _check_question_grounding(chunk_text, d, model)
651     if grounding_status == "ok":
652         accepted = True
653         accepted_question_text = d["question_text"]
654         accepted_question = d
655         break

```

```

655         break
656
657     errors.append(
658         {
659             "chunk_index": chunk_pool_idx,
660             "attempt": chunk_attempt,
661             "error": f"Grounding check failed: {grounding_reason}",
662             "data": d,
663         }
664     )
665
666     if not accepted:
667         logger.warning(
668             "Chunk %d failed after %d attempts; trying fallback chunk",
669             chunk_pool_idx,
670             _MAX_GROUNDING_RETRIES + 1,
671         )
672         if fallback_pool:
673             question_queue.append(fallback_pool.pop(0))
674         continue
675
676     timestamp = None
677     rewatch_start = None
678     end_time = meta.get("end_time")
679     start_time = meta.get("start_time")
680     if isinstance(end_time, (int, float)):
681         timestamp = float(round(end_time))
682     if isinstance(start_time, (int, float)):
683         rewatch_start = float(round(start_time))
684
685     pending_creates.append(
686         _build_pending_question_data(
687             accepted_question,
688             chunk_text,
689             [meta],
690             timestamp=timestamp,
691             rewatch_start=rewatch_start,
692         )
693     )
694     seen_question_keys.add(_normalise_question_text(accepted_question_text))
695     covered_questions.append(accepted_question_text)
696
697     return pending_creates, errors, total_latency_ms

```

8.9. Streaming chat handler and citation repair

```

340 def stream_chat_reply(payload: dict):
341     """Retrieve RAG context and stream a tutor reply progressively.
342
343     Runs a single generation attempt (no retry) and repairs the text
344     server-side if validation would have failed – keeping the user's view
345     as a single clean stream.
346     """
347     overall_start = time.monotonic()
348     requested_model = payload.get("model") or "<default>"
349     try:
350         # Tell the client we accepted the request so the spinner can flip to a
351         # live streaming state instead of appearing to hang.
352         yield _stream_event({"type": "replace", "text": ""})
353
354         # Report currently-loaded Ollama models so a long first-token delay is
355         # legible as a cold-load vs a generation stall.
356         try:
357             from tutor.services.ollama_service import list_loaded_models
358             logger.info(
359                 "[stream] start; requested_model=%s, ollama_loaded=%s",
360                 requested_model,
361                 list_loaded_models(),
362             )
363         except Exception:
364             logger.info("[stream] start; requested_model=%s (loaded-model probe failed)", requested_model)
365
366         rag_start = time.monotonic()
367         query = _build_chat_retrieval_query(payload)
368         context, metadatas = _retrieve_chat_context(query)
369         logger.info(
370             "[stream] rag retrieval: %.2fs, %d chunks kept",
371             time.monotonic() - rag_start,
372             len(context) if context else 0,
373         )
374
375         if context and metadatas:
376             citations = build_citations(metadatas)
377             citation_dicts = [c.to_dict() for c in citations]
378             grounded_prompt = build_prompt(payload, retrieval_context=context)
379             grounded_system = _build_system_prompt(SYSTEM_PROMPT, payload)
380             streamed_text = yield from _stream_one_attempt(
381                 model_name=payload.get("model"),
382                 system_prompt=grounded_system,
383                 prompt=grounded_prompt,
384                 mode="delta",
385                 phase="grounded",
386             )
387             repaired = repair_grounded_citations(streamed_text, len(citations))

```

```

387         repaired = _repair_grounded_citations(streamed_text, len(citations))
388         if repaired != streamed_text:
389             logger.info(
390                 "Post-hoc citation repair applied for session_id=%s topic=%s",
391                 payload.get("session_id"),
392                 payload.get("topic"),
393             )
394             yield _stream_event({"type": "replace", "text": repaired})
395         yield _stream_event({
396             "type": "final",
397             "text": repaired,
398             "citations": citation_dicts,
399             "latency_ms": int((time.monotonic() - overall_start) * 1000),
400         })
401         logger.info("[stream] total elapsed: %.2fs (grounded)", time.monotonic() - overall_start)
402         return
403
404     logger.warning(
405         "No usable RAG context found; using fallback for session_id=%s topic=%s",
406         payload.get("session_id"),
407         payload.get("topic"),
408     )
409
410     fallback_prompt = build_prompt(payload, retrieval_context=None)
411     fallback_system = _build_system_prompt(FALLBACK_SYSTEM_PROMPT, payload)
412     streamed_text = yield_from_stream_one_attempt(
413         model_name=payload.get("model"),
414         system_prompt=fallback_system,
415         prompt=fallback_prompt,
416         mode="delta",
417         phase="fallback",
418     )
419     cleaned = _strip_citations(streamed_text)
420     if cleaned != streamed_text:
421         logger.info(
422             "Stripped stray citations from fallback reply for session_id=%s topic=%s",
423             payload.get("session_id"),
424             payload.get("topic"),
425         )
426         yield _stream_event({"type": "replace", "text": cleaned})
427     yield _stream_event({
428         "type": "final",
429         "text": cleaned,
430         "citations": [],
431         "latency_ms": int((time.monotonic() - overall_start) * 1000),
432     })

```

```

432     })
433     logger.info("[stream] total elapsed: %.2fs (fallback)", time.monotonic() - overall_start)
434
435     except Exception:
436         logger.exception(
437             "Chat stream generation failed for session_id=%s topic=%s",
438             payload.get("session_id"),
439             payload.get("topic"),
440         )
441         yield _stream_event({
442             "type": "error",
443             "message": "Temporary backend error while generating tutor reply.",
444         })
445

```

```

234 def _stream_one_attempt(
235     *,
236     model_name: str / None,
237     system_prompt: str,
238     prompt: str,
239     mode: str,
240     phase: str = "generate",
241 ):
242     """Stream a single Ollama attempt, yielding events and returning the full text.
243
244     `mode` is "delta" (append to what's already shown) or "replace" (overwrite
245     prior partial text with the new tokens as they arrive).
246     `phase` is a log label (e.g. "grounded", "fallback") so timing output is
247     readable when the tutor falls through multiple stages.
248     """
249     accumulated = ""
250     first_chunk_in_replace = True
251     chunk_count = 0
252     start = time.monotonic()
253     logger.info("[stream:%s] opening ollama stream (model=%s)", phase, model_name or
254                 "<default>")
255
256     stream = generate_text_stream(
257         model_name=model_name,
258         system_prompt=system_prompt,
259         prompt=prompt,
260     )
261     try:
262         for chunk in stream:
263             if chunk_count == 0 and chunk.text:
264                 logger.info(
265                     "[stream:%s] first token received after %.2fs",
266                     phase,
267                     time.monotonic() - start,
268                 )
269             if chunk.done:
270                 break
271             if not chunk.text:
272                 continue
273             chunk_count += 1
274             accumulated += chunk.text
275             if mode == "delta":
276                 yield _stream_event({"type": "delta", "text": chunk.text})
277             else:
278                 if first_chunk_in_replace:
279                     yield _stream_event({"type": "replace", "text": accumulated})
280                     first_chunk_in_replace = False
281             else:

```

```

280         else:
281             yield _stream_event({"type": "delta", "text": chunk.text})
282     finally:
283         # Close the underlying Ollama HTTP response even on GeneratorExit.
284         try:
285             stream.close()
286         except Exception:
287             pass
288
289     logger.info(
290         "[stream:%s] completed in %.2fs (%d chunks, %d chars)",
291         phase,
292         time.monotonic() - start,
293         chunk_count,
294         len(accumulated),
295     )
296
297     # Strip <think>...</think> blocks the same way generate_text does.
298     cleaned = re.sub(r"<think>.*?</think>", "", accumulated, flags=re.DOTALL).strip()
299     if cleaned != accumulated:
300         # Model emitted reasoning tokens we streamed but shouldn't keep; replace.
301         yield _stream_event({"type": "replace", "text": cleaned})
302     return cleaned

```

```

305     _CITATION_PATTERN = re.compile(r"\[(\d+)\]")
306
307
308     def _repair_grounded_citations(text: str, max_index: int) -> str:
309         """Ensure the grounded response has at least one valid [N] citation.
310
311         Strategy:
312         - Drop any out-of-range [N] references (keep only 1..max_index).
313         - If none remain, append [1] to the end of the first sentence so the
314           user sees an inline citation matching the top-ranked RAG chunk.
315         """
316         def _filter(match: re.Match) -> str:
317             idx = int(match.group(1))
318             return match.group(0) if 1 <= idx <= max_index else ""
319
320         filtered = _CITATION_PATTERN.sub(_filter, text).strip()
321         if _CITATION_PATTERN.search(filtered):
322             return filtered
323
324         # No valid citation survived. Insert [1] at the end of the first sentence,
325         # or at the end of the whole reply if there's no sentence break.
326         sentence_end = re.search(r"[!?](?=\s|$)", filtered)
327         if sentence_end:
328             insert_at = sentence_end.start()
329             return f"{filtered[:insert_at]} [1]{filtered[insert_at:]}"
330         if filtered:
331             return f"{filtered} [1]"
332         return "[1]"

```

```

335     def _strip_citations(text: str) -> str:
336         """Remove any [N] citation markers from a fallback reply."""
337         return _CITATION_PATTERN.sub("", text).strip()

```

8.10. Buffered chat retry path

```
198 def get_chat_reply(payload: dict) -> ChatResponse:
199     """Retrieve RAG context and generate a tutor reply, falling back gracefully."""
200     query = _build_chat_retrieval_query(payload)
201     context, metadatas = _retrieve_chat_context(query)
202
203     if context and metadatas:
204         try:
205             return _grounded_chat_response(payload, context, metadatas)
206         except ValueError:
207             logger.warning(
208                 "Retrieved answer was not grounded; falling back for session_id=%s topic=%s",
209                 payload.get("session_id"),
210                 payload.get("topic"),
211             )
212     else:
213         logger.warning(
214             "No usable RAG context found; using fallback for session_id=%s topic=%s",
215             payload.get("session_id"),
216             payload.get("topic"),
217         )
218
219     return _fallback_chat_response(payload)
220
```

```
171 def _grounded_chat_response(payload: dict, context: list[str], metadatas: list[dict]) ->
    ChatResponse:
172     citations = build_citations(metadatas)
173     prompt = build_prompt(payload, retrieval_context=context)
174     system_prompt = _build_system_prompt(SYSTEM_PROMPT, payload)
175     text = _generate_with_retry(
176         model_name=payload.get("model"),
177         prompt=prompt,
178         system_prompt=system_prompt,
179         retry_system_prompt=f"{system_prompt}{GROUNDING_RETRY_PROMPT}",
180         validator=lambda t: response_has_valid_citations(t, len(citations)),
181     )
182     return ChatResponse(response=text, citations=citations, latency_ms=0)
```

```
185 def _fallback_chat_response(payload: dict) -> ChatResponse:
186     prompt = build_prompt(payload, retrieval_context=None)
187     system_prompt = _build_system_prompt(FALLBACK_SYSTEM_PROMPT, payload)
188     text = _generate_with_retry(
189         model_name=payload.get("model"),
190         prompt=prompt,
191         system_prompt=system_prompt,
192         retry_system_prompt=f"{system_prompt}{FALLBACK_RETRY_PROMPT}",
193         validator=response_has_no_citations,
194     )
195     return ChatResponse(response=text, citations=[], latency_ms=0)
```

```

143  def _generate_with_retry(
144      *,
145      model_name: str / None,
146      prompt: str,
147      system_prompt: str,
148      retry_system_prompt: str,
149      validator: Callable[[str], bool],
150  ) -> str:
151      """Generate text, retrying once with a correction prompt if validation fails.
152
153      Returns the validated response text, or raises ValueError if both attempts fail.
154      """
155      for system in (system_prompt, retry_system_prompt):
156          generation = generate_text(
157              model_name=model_name,
158              system_prompt=system,
159              prompt=prompt,
160          )
161          if validator(generation.text):
162              return generation.text
163
164      raise ValueError("LLM reply failed validation after retry.")
165

```

```

54  # Appended to SYSTEM_PROMPT on retry when the first attempt contained no valid citations.
55  GROUNDING_RETRY_PROMPT = """\
56
57  ## Grounding correction
58  - Your previous draft was rejected because it was not explicitly grounded in the retrieved
    context.
59  - Revise the answer so it uses the retrieved chunks directly.
60  - Include at least one valid inline citation marker like [1] or [2].
61  - Every citation marker must refer to one of the provided retrieved chunks only.\
62  """

```

```

88  # Appended to FALLBACK_SYSTEM_PROMPT on retry when the first attempt included citations.
89  FALLBACK_RETRY_PROMPT = """\
90
91  ## Fallback correction
92  - Your previous draft was rejected because fallback replies must never include citations or
    source markers.
93  - If the message is relevant, start with exactly "Slightly beyond the video:" and keep it to 2
    sentences maximum.
94  - If the message is not relevant, politely refuse in 1 sentence and ask the student to
    rephrase or return to the video topic.\
95  """

```

8.11. SEP sources json

```

1  {
2  {
3      "id": "liberalism",
4      "url": "https://plato.stanford.edu/entries/liberalism/",
5      "title": "Liberalism",
6      "file": "liberalism.html",
7      "accessed": "2026-04-16"
8  },
9  {
10     "id": "mill-moral-political",
11     "url": "https://plato.stanford.edu/entries/mill-moral-political/",
12     "title": "Mill's Moral and Political Philosophy",
13     "file": "mill-moral-political.html",
14     "accessed": "2026-04-16"
15 },
16 {
17     "id": "mill",
18     "url": "https://plato.stanford.edu/entries/mill/",
19     "title": "John Stuart Mill",
20     "file": "mill.html",
21     "accessed": "2026-04-16"
22 },
23 {
24     "id": "democracy",
25     "url": "https://plato.stanford.edu/entries/democracy/",
26     "title": "Democracy",
27     "file": "democracy.html",
28     "accessed": "2026-04-16"
29 },
30 {
31     "id": "rousseau",
32     "url": "https://plato.stanford.edu/entries/rousseau/",
33     "title": "Jean Jacques Rousseau",
34     "file": "rousseau.html",
35     "accessed": "2026-04-16"
36 }

```

8.12. Consent form

PARTICIPANT CONSENT FORM

Title of Study: A popup chatbot tutor to scaffold learning from video lectures and assess effectiveness

Lead researcher: Hamzah Patel (22468473@stu.mmu.ac.uk)

Participant Identification Number for this project: _____

	Please initial
1. I confirm that I have read and understood the information sheet explaining the above research project and I have had the opportunity to ask questions about the project.	
2. I understand that my participation is voluntary and that I am free to withdraw at any time without giving any reason and without any negative consequences. In addition, should I not wish to answer any particular question or questions, I am free to decline.	
3. I am happy for my responses to be analysed.	
4. I understand that my responses will be anonymous. I give permission for members of the research team to have access to my anonymised responses. I understand that my name will not be linked with the research materials, and I will not be identified or identifiable in the report or reports that result from the research.	
5. I understand that my taking part is voluntary and that I can withdraw from the study at any time/before 28/03/2026; I do not have to give any reasons for why I no longer want to take part and there will be no adverse consequences if I choose to withdraw.	
6. I agree to take part in the above research project. I understand that taking part in the project will involve completing 2 questionnaires/quizzes, watching 2 videos and interacting with an AI tutor during 1 video.	
How my information will be used during and after the project	
I understand my personal details such as name, phone number, address and email address etc. will not be revealed to people outside the project.	
I understand and agree that my words may be quoted in publications, reports, web pages, and other research outputs. I understand that I will not be named in these outputs unless I specifically request this.	

I understand and agree that other authorised researchers will have access to this data only if they agree to preserve the confidentiality of the information as requested in this form.

I understand and agree that other authorised researchers may use my data in publications, reports, web pages, and other research outputs, only if they agree to preserve the confidentiality of the information as requested in this form.

I give permission for the questionnaires conducted during the experiment, and data relating to my AI interactions to be deposited in One Drive so it can be used for future research and learning.

So that the information you provide can be used legally by the researchers

I agree to assign the copyright I hold in any materials generated as part of this project to Manchester Metropolitan University

Name of Participant

Date

Signature

Lead Researcher

Date

Signature

To be signed and dated in presence of the participant.

8.13. Participant Information Sheet

Participant Information Sheet

My name is Hamzah Patel and I am studying for a BSc (Hons) Computer Science degree at Manchester Metropolitan University (22468473@stu.mmu.ac.uk). I am undertaking my final year project called 'A popup chatbot tutor to scaffold learning from video lectures and assess effectiveness' and I would like you to take part in a study.

Before you decide whether to take part, it is important that you understand what you will be required to do. Please take the time to read the following information and feel free to ask if anything is not clear or if you need more information.

What is the purpose of the study?

The purpose of the study is to assess the effectiveness of an AI chatbot tutor designed to assist you with video learning. It will utilise retrieval augmented generation to prevent against hallucinations alongside a large language model (similar to ChatGPT) to help guide the learner towards learning.

Do I have to take part?

No. It is up to you to decide whether or not to take part. If you do decide to take part, you will be asked to sign a consent form. However, you will still be free to withdraw at any time and without giving a reason.

What will happen to me if I take part?

You will first read this participation sheet and have the opportunity to ask questions.

If you agree to take part, you will then sign a consent form.

We would like you to fill in a pre-video quiz, try to use an AI chatbot tutor designed to assist with learning via video lectures for one video, watch a separate video without the tutor assistance and then fill in a post-video quiz to assess the effectiveness of the AI tutor when compared with the control video (the video without the AI tutor).

Your quiz form responses, your chat logs and telemetry data such as how long you spent talking to the LLM and how long you spent thinking/reading between responses will be recorded during the experiment. You will be able to communicate with the experimenter during the experiment if you need to do so. If you are uncomfortable or have a problem, it is possible to take a break or stop the experiment altogether at any time.

Once the study is completed, you will be debriefed, and may ask any questions about the study or the procedure.

What are the possible disadvantages and risks of taking part?

There are no known risks associated with the study.

Will my taking part in this study be kept confidential?

Yes. All information will be treated in strict confidence. Data will be stored on password protected computers and only the research team will have access to the data. Your test results will not be revealed to anyone, and will be stored anonymised and separate from any personal data like your name.

What will happen to the results of the research study?

The findings of this study will be used in my project report and presentation and may be used in scientific publications and conferences. However, we will keep your name and personal details confidential, and we will ensure that you cannot be identified in these reports.

Please note that new data protection legislation has come into effect across the UK; this means that we need to provide you with some further information relating to how your personal information will be used and managed within this research project.

Manchester Metropolitan University will act as the Data Controller for this study. This means that the University is responsible for looking after your information and using it properly. In order to collect and use your personal information as part of this research project, we must have a basis in law to do so. The basis that we are using is that the research is 'a task in the public interest'.

Further information, including details about how and why the University processes your personal information, how we keep your information secure, and your legal rights (including how to complain if you feel that your personal information has not been handled correctly), can be found here: <https://www.mmu.ac.uk/data-protection/dppolicy>

Contact for further information

If you require any further information, please contact Hamzah Patel, School of Computing, Mathematics and Digital Technology, email: 22468473@stu.mmu.ac.uk.

Alternatively, you may contact the research supervisor Keeley Crockett (K.Crockett@mmu.ac.uk), or the module leader Nicholas Costen (N.Costen@mmu.ac.uk).

Alternatively, you may contact the research supervisor Keeley Crockett (K.Crockett@mmu.ac.uk), or the Module leader Nicholas Costen (N.Costen@mmu.ac.uk).

Further information can be obtained from the Manchester Metropolitan Data Protection Officer our Data Protection Officer can be contacted using the dataprotection@mmu.ac.uk e-mail address, by calling +44 (0)7584 330586 or in writing to: Data Protection Officer, Legal & Governance, Ormond Building, Lower Ormond Street, Manchester, M15 6BX.

8.14. Anonymised Likert questionnaire responses

Participant	Session ID	Hints helpful	Difficulty appropriate	Disruption	Preferred AI tutor	Response clarity
A	33799	Strongly Agree	Strongly Agree	Disagree	Strongly Agree	Neither agree nor disagree
B	29917	Agree	Agree	Strongly disagree	Strongly Agree	Strongly Agree
C	70982	Agree	Strongly Agree	Strongly disagree	Strongly Agree	Agree
D	33383	Agree	Agree	Strongly disagree	Strongly Agree	Agree
E	28996	Strongly Agree	Strongly Agree	Disagree	Strongly Agree	Strongly Agree
F	76878	Agree	Agree	Disagree	Strongly Agree	Strongly Agree

8.15. Anonymised free-text questionnaire responses

Participant	Session ID	Most helpful	Frustrating or confusing	Anything else
A	33799	It promotes engagement rather than passively watching at the risk of zoning out.	When engaging with questions of the tutor by chat the reply did not always clearly indicate whether my answer was correct or not.	Good job.
B	29917	Good level of explanation provided with each response. No big words were used which made it easy to understand	Nothing	I'd like to see this model incorporated into university courses for self-study and revision for exams.
C	70982	I like how it already had context about the video i was watching so it could help me understand.	The tutor would ask me for an example sometimes or to expand on my answer and didn't really tell me if I was entirely correct	I would like to see this integrated into Moodle so it could help me with revising and understanding concepts.
D	33383	It forced me to engage with the video instead of autopiloting and just feeling like I was learning	Sometimes the questions in the chat felt confusing as it felt there were multiple correct answers	Its a great proof of concept but the LLM feels underpowered compared to something like Claude
E	28996	The fact that if I made a mistake when answering, a new question would appear and I had the choice to be asked more questions until I felt ready.	Keeping in all the information from the video without having the breaks from the AI tutor.	No.
F	76878	The hints and follow up explanations	Certain language used in the videos	n/a

Participant labels A–F are anonymised labels used for reporting questionnaire responses in Section 5.6. Microsoft Forms metadata and identifying fields were removed before inclusion.